



Linka

Data: 11/10/2025

Este documento contém a estrutura de pastas e todos os códigos do projeto, gerado automaticamente.

Estrutura de Pastas


```

├── .env
├── .env.example
├── .husky/
│   ├── _/
│   │   ├── applypatch-msg
│   │   ├── commit-msg
│   │   ├── h
│   │   ├── husky.sh
│   │   ├── post-applypatch
│   │   ├── post-checkout
│   │   ├── post-commit
│   │   ├── post-merge
│   │   ├── post-rewrite
│   │   ├── pre-applypatch
│   │   ├── pre-auto-gc
│   │   ├── pre-commit
│   │   ├── pre-merge-commit
│   │   ├── pre-push
│   │   ├── pre-rebase
│   │   └── prepare-commit-msg
│   ├── commit-msg
│   └── pre-commit
├── .prettierrc.json
├── App.tsx
├── LICENSE
├── README.md
├── app.json
├── assets/
│   ├── adaptive-icon.png
│   ├── favicon.png
│   ├── icon.png
│   └── splash-icon.png
├── codigo.md
├── commitlint.config.js
├── eas.json
├── eslint.config.mjs
└── gerar_codigo.py

```

```
├── index.ts
├── package.json
├── pnpm-lock.yaml
├── src/
│   ├── assets/
│   │   └── logo.png
│   ├── components/
│   │   ├── LocationSelectorModal/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── PristineSearch/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── ReviewModal/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   └── ServiceCard/
│   │       ├── index.tsx
│   │       └── style.ts
│   ├── hooks/
│   │   └── useDebounce.ts
│   ├── navigation/
│   │   ├── AppNavigator.tsx
│   │   ├── AuthNavigator.tsx
│   │   ├── TabNavigator.tsx
│   │   └── types.ts
│   ├── screens/
│   │   ├── addService/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── changePassword/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── chat/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── completeProfile/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── confirmEmail/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── conversations/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── editProfile/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   ├── forgotPassword/
│   │   │   ├── index.tsx
│   │   │   └── style.ts
│   │   └── home/
│   │       ├── index.tsx
│   │       └── style.ts
```

```
login/
├── index.tsx
├── style.ts
├── profile/
│   ├── index.tsx
│   └── style.ts
├── register/
│   ├── index.tsx
│   └── style.ts
├── resetPassword/
│   ├── index.tsx
│   └── style.ts
├── search/
│   ├── index.tsx
│   └── style.ts
├── serviceDetail/
│   ├── index.tsx
│   └── style.ts
├── welcome/
│   ├── index.tsx
│   └── style.ts
└── services/
    └── supabase.ts
tsconfig.json
```

📁 Códigos do Projeto

📁 ./App.tsx

```
import React, { useState, useEffect } from 'react';
import { Session } from '@supabase/supabase-js';
import { NavigationContainer, LinkingOptions } from '@react-navigation/native';
import { View, ActivityIndicator, Text } from 'react-native';
import * as Linking from 'expo-linking';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import { supabase } from '../src/services/supabase';
import AuthNavigator from '../src/navigation/AuthNavigator';
import AppNavigator from '../src/navigation/AppNavigator';
import { AuthStackParamList, AppStackParamList } from '../src/navigation/types';

const prefix = Linking.createURL('/');

/**
 * @description
 * O componente raiz do aplicativo.
 * Suas responsabilidades são:
 * 1. Prover o contexto de Área Segura para toda a aplicação.
 * 2. Gerenciar o estado da sessão de autenticação.
 * 3. Configurar o deep linking para fluxos como a recuperação de senha.
```

```
*/
export default function App() {
  const [session, setSession] = useState<Session | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setLoading(false);
    });

    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
      if (
        _event === 'SIGNED_OUT' ||
        _event === 'SIGNED_IN' ||
        _event === 'PASSWORD_RECOVERY'
      ) {
        setLoading(false);
      }
    });

    return () => subscription.unsubscribe();
  }, []);

  /**
   * @description
   * Configuração do Deep Linking para o React Navigation.
   * Conecta URLs (como as de e-mail) a telas específicas do aplicativo.
   */
  const linking: LinkingOptions<AuthStackParamList & AppStackParamList> = {
    prefixes: [prefix],
    config: {
      screens: {
        ResetPassword: 'reset-password',
      },
    },
  };

  if (loading) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
        <ActivityIndicator size="large" color="#3F83F8" />
      </View>
    );
  }

  return (
    <SafeAreaProvider>
      <NavigationContainer
        linking={linking}

```

```
      fallback={<Text>Carregando...</Text>}  
    >  
      {session && session.user ? <AppNavigator /> : <AuthNavigator />}  
    </NavigationContainer>  
  </SafeAreaProvider>  
);  
}
```

📁 ./commitlint.config.js

```
module.exports = {  
  extends: ['@commitlint/config-conventional'],  
};
```

📁 ./index.ts

```
import { registerRootComponent } from 'expo';  
  
import App from './App';  
  
// registerRootComponent calls AppRegistry.registerComponent('main', () =>  
App);  
// It also ensures that whether you load the app in Expo Go or in a native  
build,  
// the environment is set up appropriately  
registerRootComponent(App);
```

📁 ./github/workflows/main.yml

```
name: Verificação de Qualidade do Código  
on: [push, pull_request]  
  
permissions:  
  contents: read  
  
jobs:  
  prettier:  
    name: Verificação de Formatação (Prettier)  
    runs-on: ubuntu-latest  
    steps:  
      - name: 1. Baixar o código do repositório  
        uses: actions/checkout@v4
```

- name: 2. Configurar PNPM
uses: pnpm/action-setup@v4
with:
 version: 9
- name: 3. Configurar ambiente Node.js
uses: actions/setup-node@v4
with:
 node-version: '20'
 cache: 'pnpm'
- name: 4. Instalar dependências
run: pnpm install --frozen-lockfile
- name: 5. Rodar a verificação do Prettier
run: pnpm run lint:prettier:check

eslint:

name: Verificação de Código (ESLint)
runs-on: ubuntu-latest

steps:

- name: 1. Baixar o código do repositório
uses: actions/checkout@v4
- name: 2. Configurar PNPM
uses: pnpm/action-setup@v4
with:
 version: 9
- name: 3. Configurar ambiente Node.js
uses: actions/setup-node@v4
with:
 node-version: '20'
 cache: 'pnpm'
- name: 4. Instalar dependências
run: pnpm install --frozen-lockfile
- name: 5. Rodar a verificação do ESLint
run: pnpm run lint

commitlint:

name: Verificação de Commits (Commitlint)
runs-on: ubuntu-latest

if: github.event_name == 'push'

steps:

- name: 1. Baixar o código do repositório
uses: actions/checkout@v4
with:
 fetch-depth: 0
- name: 2. Configurar PNPM
uses: pnpm/action-setup@v4
with:

```
    version: 9

  - name: 3. Configurar ambiente Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '20'
      cache: 'pnpm'

  - name: 4. Instalar dependências
    run: pnpm install --frozen-lockfile

  - name: 5. Rodar a verificação do Commitlint
    run: pnpm exec commitlint --last
```

📁 ./src/services/supabase.ts

```
import 'react-native-url-polyfill/auto';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY;

if (!supabaseUrl || !supabaseAnonKey) {
  throw new Error(
    'As variáveis de ambiente do Supabase não foram definidas. Verifique seu arquivo .env',
  );
}

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: AsyncStorage,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false,
  },
});
```

📁 ./src/screens/home/style.ts

```
import { StyleSheet, Platform, Dimensions } from 'react-native';

const { width: SCREEN_WIDTH } = Dimensions.get('window');
const CARD_WIDTH = SCREEN_WIDTH * 0.42;
const HORIZONTAL_PADDING = 20;
```

```
export const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },

  loadingContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#FFFFFF',
  },

  loadingText: {
    marginTop: 12,
    fontSize: 14,
    color: '#64748B',
    fontWeight: '500',
  },

  // Header Styles
  header: {
    backgroundColor: '#FFFFFF',
    paddingHorizontal: HORIZONTAL_PADDING,
    paddingTop: 16,
    paddingBottom: 20,
    borderBottomWidth: 1,
    borderBottomColor: '#F1F5F9',
    ...Platform.select({
      ios: {
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.05,
        shadowRadius: 3,
      },
      android: {
        elevation: 2,
      },
    }),
  },

  headerContent: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },

  headerTextContainer: {
    flex: 1,
    marginRight: 12,
  },

  greeting: {
    fontSize: 24,
```



```
    fontWeight: '700',
    color: '#0F172A',
    letterSpacing: -0.5,
  },

  subtitle: {
    fontSize: 14,
    color: '#64748B',
    marginTop: 4,
    fontWeight: '400',
  },

  addButton: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#3B82F6',
    paddingHorizontal: 16,
    paddingVertical: 10,
    borderRadius: 12,
    gap: 6,
    ...Platform.select({
      ios: {
        shadowColor: '#3B82F6',
        shadowOffset: { width: 0, height: 4 },
        shadowOpacity: 0.2,
        shadowRadius: 8,
      },
      android: {
        elevation: 4,
      },
    }),
  },

  addButtonText: {
    color: 'FFFFFF',
    fontSize: 14,
    fontWeight: '600',
  },

  // ScrollView
  scrollView: {
    flex: 1,
    backgroundColor: '#F8FAFC',
  },

  // Section Styles
  section: {
    marginTop: 24,
  },

  lastSection: {
    marginBottom: 24,
  },
```

```
sectionHeader: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  paddingHorizontal: HORIZONTAL_PADDING,
  marginBottom: 16,
},

sectionTitleRow: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 8,
},

sectionTitle: {
  fontSize: 18,
  fontWeight: '600',
  color: '#0F172A',
  letterSpacing: -0.3,
},

viewAllText: {
  fontSize: 14,
  fontWeight: '600',
  color: '#3B82F6',
},

// Categories
categoriesContainer: {
  paddingHorizontal: HORIZONTAL_PADDING,
  gap: 12,
},

categoryCard: {
  alignItems: 'center',
  marginRight: 12,
  width: 72,
},

categoryIconWrapper: {
  width: 56,
  height: 56,
  borderRadius: 16,
  justifyContent: 'center',
  alignItems: 'center',
  marginBottom: 8,
},

categoryLabel: {
  fontSize: 12,
  color: '#475569',
  fontWeight: '500',
  textAlign: 'center',
},
```

```
// Services
servicesContainer: {
  paddingHorizontal: HORIZONTAL_PADDING,
  gap: 12,
},

serviceCardWrapper: {
  width: CARD_WIDTH,
  marginRight: 12,
},

// Empty State
emptyContainer: {
  alignItems: 'center',
  justifyContent: 'center',
  paddingVertical: 32,
  paddingHorizontal: HORIZONTAL_PADDING,
},

emptyText: {
  marginTop: 12,
  fontSize: 14,
  color: '#94A3B8',
  textAlign: 'center',
},
});
```

📁 ./src/screens/home/index.tsx

```
import React, { useState, useCallback } from 'react';
import {
  Text,
  TouchableOpacity,
  View,
  ScrollView,
  ActivityIndicator,
  Alert,
  FlatList,
  StatusBar,
  RefreshControl,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import {
  useNavigation,
  useFocusEffect,
  CommonActions,
} from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { AppStackParamList } from '../../../navigation/types';
import { supabase } from '../../../services/supabase';
```

```

import { styles } from './style';
import { Ionicons } from '@expo/vector-icons';
import ServiceCard, { ServiceCardData } from
'../../components/ServiceCard';

type HomeScreenNavigationProp = NativeStackNavigationProp<
  AppStackParamList,
  'MainTabs'
>;

interface Category {
  id: number;
  name: string;
  icon: keyof typeof Ionicons.glyphMap;
  color: string;
}

/**
 * Tela inicial do aplicativo LINKA
 * Conecta prestadores de serviços locais com clientes da comunidade
 */
const HomeScreen: React.FC = () => {
  const navigation = useNavigation<HomeScreenNavigationProp>();
  const [isLoading, setIsLoading] = useState(true);
  const [isRefreshing, setIsRefreshing] = useState(false);
  const [userName, setUserName] = useState('');
  const [topRatedServices, setTopRatedServices] =
useState<ServiceCardData[]>([]);
  const [recentServices, setRecentServices] = useState<ServiceCardData[]>
([]);

  const categories: Category[] = [
    { id: 1, name: 'Reparos', icon: 'hammer-outline', color: '#3B82F6' },
    { id: 2, name: 'Aulas', icon: 'school-outline', color: '#8B5CF6' },
    { id: 3, name: 'Cuidados', icon: 'heart-outline', color: '#EC4899' },
    { id: 4, name: 'Jardinagem', icon: 'leaf-outline', color: '#10B981' },
    { id: 5, name: 'Tecnologia', icon: 'laptop-outline', color: '#F59E0B' },
  ],
  { id: 6, name: 'Transporte', icon: 'car-outline', color: '#6366F1' },
  ];

  /**
   * Carrega dados do usuário e serviços
   */
  const loadData = async (showLoader = true) => {
    try {
      if (showLoader) setIsLoading(true);

      const { data: { user } } = await supabase.auth.getUser();
      if (!user) throw new Error('Usuário não autenticado');

      const [profileData, topServicesData, recentServicesData] = await
Promise.all([
        supabase

```

```
        .from('profiles')
        .select('full_name')
        .eq('id', user.id)
        .single(),
    supabase.rpc('get_top_rated_services', { limit_count: 4 }),
    supabase
        .from('services')
        .select('*')
        .order('created_at', { ascending: false })
        .limit(4),
  ]);

  if (profileData.error) throw profileData.error;

  const firstName = profileData.data?.full_name?.split(' ')[0] ||
'Usuário';
  setUsername(firstName);
  setTopRatedServices(topServicesData.data || []);
  setRecentServices(recentServicesData.data || []);
} catch (error) {
  const errorMessage = error instanceof Error
    ? error.message
    : 'Erro ao carregar informações';
  Alert.alert('Erro', errorMessage);
} finally {
  setIsLoading(false);
  setIsRefreshing(false);
}
};

/**
 * Atualiza dados ao focar na tela
 */
useFocusEffect(
  useCallback(() => {
    loadData();
  }, [])
);

/**
 * Função de refresh pull-to-refresh
 */
const handleRefresh = () => {
  setIsRefreshing(true);
  loadData(false);
};

/**
 * Navega para tela de pesquisa com categoria selecionada
 */
const handleCategoryPress = (categoryId: number) => {
  navigation.dispatch(
    CommonActions.navigate({
      name: 'Pesquisar',
```

```
        params: { categoryId },
      } as any)
    );
  };

  /**
   * Navega para adicionar novo serviço
   */
  const handleAddService = () => {
    navigation.navigate('AddService');
  };

  /**
   * Renderiza item de categoria
   */
  const renderCategory = ({ item }: { item: Category }) => (
    <TouchableOpacity
      style={styles.categoryCard}
      onPress={() => handleCategoryPress(item.id)}
      activeOpacity={0.7}
    >
      <View style={[styles.categoryIconWrapper, { backgroundColor:
`$${item.color}15` }]}>
        <Ionicons name={item.icon} size={24} color={item.color} />
      </View>
      <Text style={styles.categoryLabel} numberOfLines={1}>
        {item.name}
      </Text>
    </TouchableOpacity>
  );

  /**
   * Renderiza card de serviço
   */
  const renderServiceCard = ({ item }: { item: ServiceCardData }) => (
    <View style={styles.serviceCardWrapper}>
      <ServiceCard service={item} />
    </View>
  );

  /**
   * Renderiza seção vazia
   */
  const renderEmptySection = (message: string) => (
    <View style={styles.emptyContainer}>
      <Ionicons name="cube-outline" size={48} color="#CBD5E1" />
      <Text style={styles.emptyText}>{message}</Text>
    </View>
  );

  if (isLoading) {
    return (
      <View style={styles.loadingContainer}>
        <ActivityIndicator size="large" color="#3B82F6" />
      </View>
    );
  }
}
```

```
        <Text style={styles.loadingText}>Carregando...</Text>
      </View>
    );
  }

  return (
    <SafeAreaView style={styles.container} edges={['top']}>
      <StatusBar barStyle="dark-content" backgroundColor="#FFFFFF" />

      {/* Header */}
      <View style={styles.header}>
        <View style={styles.headerContent}>
          <View style={styles.headerTextContainer}>
            <Text style={styles.greeting}>Olá, {userName} 🖐️</Text>
            <Text style={styles.subtitle}>
              Encontre serviços na sua comunidade
            </Text>
          </View>
          <TouchableOpacity
            style={styles.addButton}
            onPress={handleAddService}
            activeOpacity={0.8}
          >
            <Ionicons name="add" size={20} color="#FFFFFF" />
            <Text style={styles.addButtonText}>Adicionar</Text>
          </TouchableOpacity>
        </View>
      </View>

      <ScrollView
        style={styles.scrollView}
        showsVerticalScrollIndicator={false}
        refreshControl={
          <RefreshControl
            refreshing={isRefreshing}
            onRefresh={handleRefresh}
            colors={['#3B82F6']}
            tint color="#3B82F6"
          />
        }
      >
        {/* Categorias */}
        <View style={styles.section}>
          <View style={styles.sectionHeader}>
            <Text style={styles.sectionTitle}>Categorias</Text>
            <TouchableOpacity onPress={() => navigation.dispatch(
              CommonActions.navigate({ name: 'Pesquisar' } as any)
            )}>
              <Text style={styles.viewAllText}>Ver todas</Text>
            </TouchableOpacity>
          </View>
          <FlatList
            data={categories}
            renderItem={renderCategory}
          />
        </View>
      </ScrollView>
    </SafeAreaView>
  );
}
```

```

        keyExtractor={({item}) => item.id.toString()}
        horizontal
        showsHorizontalScrollIndicator={false}
        contentContainerStyle={styles.categoriesContainer}
      />
</View>

{/* Serviços em Destaque */}
<View style={styles.section}>
  <View style={styles.sectionHeader}>
    <View style={styles.sectionTitleRow}>
      <Icons name="star" size={20} color="#F59E0B" />
      <Text style={styles.sectionTitle}>Mais Bem Avaliados</Text>
    </View>
    <TouchableOpacity onPress={() => navigation.dispatch(
      CommonActions.navigate({ name: 'Pesquisar' } as any)
    )}>
      <Text style={styles.viewAllText}>Ver todos</Text>
    </TouchableOpacity>
  </View>
  {topRatedServices.length > 0 ? (
    <FlatList
      data={topRatedServices}
      renderItem={renderServiceCard}
      keyExtractor={({item}) => item.id.toString()}
      horizontal
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={styles.servicesContainer}
    />
  ) : (
    renderEmptySection('Nenhum serviço avaliado ainda')
  )}
</View>

{/* Serviços Recentes */}
<View style={[styles.section, styles.lastSection]}>
  <View style={styles.sectionHeader}>
    <View style={styles.sectionTitleRow}>
      <Icons name="time-outline" size={20} color="#6366F1" />
      <Text style={styles.sectionTitle}>Adicionados
Recentemente</Text>
    </View>
    <TouchableOpacity onPress={() =>
navigation.navigate('Pesquisar' as any)}>
      <Text style={styles.viewAllText}>Ver todos</Text>
    </TouchableOpacity>
  </View>
  {recentServices.length > 0 ? (
    <FlatList
      data={recentServices}
      renderItem={renderServiceCard}
      keyExtractor={({item}) => item.id.toString()}
      horizontal
      showsHorizontalScrollIndicator={false}

```



```
        contentContainerStyle={styles.servicesContainer}
      />
    ) : (
      renderEmptySection('Nenhum serviço recente')
    )}
  </View>
</ScrollView>
</SafeAreaView>
);
};

export default HomeScreen;
```

📁 ./src/screens/serviceDetail/style.ts

```
import { StyleSheet, Dimensions } from 'react-native';

const { width } = Dimensions.get('window');

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },
  loadingContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  scrollContainer: {
    flexGrow: 1,
    backgroundColor: '#FFFFFF',
  },
  image: {
    width: width,
    height: 250,
    resizeMode: 'cover',
  },
  imageCounter: {
    position: 'absolute',
    bottom: 10,
    right: 10,
    backgroundColor: 'rgba(0, 0, 0, 0.6)',
    color: '#FFFFFF',
    paddingHorizontal: 10,
    paddingVertical: 5,
    borderRadius: 15,
    fontSize: 12,
    overflow: 'hidden',
  },
  contentContainer: {
```

```
padding: 20,
},
categoryTag: {
  backgroundColor: '#E6F0FF',
  color: '#3F83F8',
  paddingHorizontal: 12,
  paddingVertical: 6,
  borderRadius: 15,
  alignSelf: 'flex-start',
  overflow: 'hidden',
  marginBottom: 12,
  fontWeight: '500',
  fontSize: 14,
},
title: {
  fontSize: 28,
  fontWeight: 'bold',
  color: '#1A202C',
  marginBottom: 16,
},
providerContainer: {
  flexDirection: 'row',
  alignItems: 'center',
  padding: 16,
  backgroundColor: '#F7FAFC',
  borderRadius: 12,
  borderWidth: 1,
  borderColor: '#E2E8F0',
},
providerAvatar: {
  width: 50,
  height: 50,
  borderRadius: 25,
  marginRight: 15,
  backgroundColor: '#E2E8F0',
},
providerName: {
  fontSize: 18,
  fontWeight: 'bold',
  color: '#2D3748',
},
sectionTitle: {
  fontSize: 20,
  fontWeight: 'bold',
  color: '#1A202C',
  marginTop: 24,
  marginBottom: 12,
},
description: {
  fontSize: 16,
  color: '#4A5568',
  lineHeight: 26,
},
reviewsHeader: {
```

```
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
    paddingBottom: 8,
  },
  addReviewButton: {
    backgroundColor: 'transparent',
  },
  addReviewButtonText: {
    color: '#3F83F8',
    fontWeight: 'bold',
    fontSize: 16,
  },
  noReviewsText: {
    textAlign: 'center',
    marginTop: 20,
    color: '#666',
  },
  reviewItem: {
    paddingVertical: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
  },
  reviewHeader: {
    flexDirection: 'row',
    alignItems: 'center',
    marginBottom: 8,
  },
  reviewAvatar: {
    width: 40,
    height: 40,
    borderRadius: 20,
    marginRight: 12,
    backgroundColor: '#E2E8F0',
  },
  reviewAuthor: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#2D3748',
  },
  reviewStars: {
    flexDirection: 'row',
    marginLeft: 'auto',
  },
  reviewComment: {
    fontSize: 15,
    color: '#4A5568',
    lineHeight: 22,
  },
  footer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
```

```
    alignItems: 'center',
    paddingHorizontal: 20,
    paddingVertical: 15,
    borderTopWidth: 1,
    borderTopColor: '#E2E8F0',
    backgroundColor: '#FFFFFF',
    paddingBottom: 25,
  },
  priceLabel: {
    fontSize: 14,
    color: '#718096',
  },
  priceValue: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#1A202C',
  },
  actionButton: {
    backgroundColor: '#3F83F8',
    paddingVertical: 16,
    paddingHorizontal: 30,
    borderRadius: 12,
  },
  actionButtonText: {
    color: '#FFFFFF',
    fontSize: 16,
    fontWeight: 'bold',
  },
},
});
```

📁 ./src/screens/serviceDetail/index.tsx

```
import React, { useState, useCallback } from 'react';
import {
  View,
  Text,
  SafeAreaView,
  Image,
  Alert,
  ActivityIndicator,
  FlatList,
  ScrollView,
  TouchableOpacity,
  StatusBar,
  Dimensions,
} from 'react-native';
import {
  useRoute,
  RouteProp,
  useFocusEffect,
```

```

    useNavigation,
  } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { supabase } from '../../services/supabase';
import type { AppStackParamList } from '../../navigation/types';
import { styles } from './style';
import ReviewModal from '../../components/ReviewModal';
import { Ionicons } from '@expo/vector-icons';

type ProviderProfile = {
  id: string;
  full_name: string;
  avatar_url: string;
};

type ReviewData = {
  id: number;
  rating: number;
  comment: string | null;
  profiles: { full_name: string; avatar_url: string } | null;
};

type ServiceDetails = {
  id: number;
  title: string;
  description: string | null;
  price: number | null;
  photo_urls: string[] | null;
  availability: string | null;
  profiles: ProviderProfile | null;
  categories: { name: string } | null;
  reviews: ReviewData[];
};

type ServiceDetailRouteProp = RouteProp<AppStackParamList,
'ServiceDetail'>;
type ServiceDetailNavigationProp =
NativeStackNavigationProp<AppStackParamList>;

// --- Custom Hook ---
const useServiceDetails = (serviceId: number) => {
  const navigation = useNavigation<ServiceDetailNavigationProp>();
  const [service, setService] = useState<ServiceDetails | null>(null);
  const [loading, setLoading] = useState(true);

  const fetchServiceDetails = useCallback(async () => {
    if (!serviceId) return;
    setLoading(true);
    try {
      const { data, error } = await supabase
        .from('services')
        .select(
          `*, profiles(*), categories(name), reviews(id, rating, comment,
profiles(full_name, avatar_url))`,
        )
        .eq('id', serviceId)
        .single();
    }
  });

```

```
        if (error) throw error;
        setService(data as ServiceDetails);
    } catch (error) {
        const message =
            error instanceof Error
                ? error.message
                : 'Não foi possível carregar os detalhes.';
        Alert.alert('Erro', message);
    } finally {
        setLoading(false);
    }
}, [serviceId]);

useFocusEffect(
    useCallback(() => {
        fetchServiceDetails();
    }, [fetchServiceDetails]),
);

const handleSubmitReview = async (rating: number, comment: string) => {
    try {
        const {
            data: { user },
        } = await supabase.auth.getUser();
        if (!user) throw new Error('Você precisa estar logado para avaliar.');
```

avaliar.');

```
        if (user.id === service?.profiles?.id) {
            Alert.alert(
                'Ação inválida',
                'Você não pode avaliar o seu próprio serviço.',
            );
            return;
        }

        const { error } = await supabase
            .from('reviews')
            .insert({ service_id: serviceId, user_id: user.id, rating, comment
});

        if (error) throw error;

        Alert.alert('Sucesso', 'Sua avaliação foi enviada!');
        fetchServiceDetails();
    } catch (error) {
        const message =
            error instanceof Error
                ? error.message
                : 'Ocorreu um erro desconhecido.';
        Alert.alert('Erro ao avaliar', message);
    }
};

const handleStartConversation = async () => {
    if (!service?.profiles?.id) return;
```

```
try {
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user)
    throw new Error('Você precisa estar logado para iniciar uma conversa.');
```

```
  if (user.id === service.profiles.id) {
    Alert.alert(
      'Ação inválida',
      'Você não pode iniciar uma conversa consigo mesmo.',
    );
    return;
  }

  const { data: conversationId, error } = await supabase.rpc(
    'get_or_create_conversation',
    {
      participant1_id_input: user.id,
      participant2_id_input: service.profiles.id,
    },
  );
  if (error) throw error;

  if (conversationId && service.profiles) {
    navigation.navigate('Chat', {
      conversationId,
      recipient: service.profiles,
    });
  }
} catch (error) {
  const message =
    error instanceof Error
      ? error.message
      : 'Ocorreu um erro desconhecido.';
  Alert.alert('Erro ao iniciar conversa', message);
}
};
return { service, loading, handleSubmitReview, handleStartConversation };
};

const ReviewItem: React.FC<{ review: ReviewData }> = ({ review }) => (
  <View style={styles.reviewItem}>
    <View style={styles.reviewHeader}>
      <Image
        source={{ uri: review.profiles?.avatar_url }}
        style={styles.reviewAvatar}
      />
      <Text style={styles.reviewAuthor}>
        {review.profiles?.full_name || 'Usuário Anônimo'}
      </Text>
      <View style={styles.reviewStars}>
        [...Array(5)].map((_, i) => (
          <Ionicons
```

```

        key={i}
        name="star"
        size={16}
        color={i < review.rating ? '#FFC107' : '#CCC'}
      />
    )}}
  </View>
</View>
{review.comment && (
  <Text style={styles.reviewComment}>{review.comment}</Text>
)}
</View>
);

const ServiceDetailScreen: React.FC = () => {
  const route = useRoute<ServiceDetailRouteProp>();
  const { serviceId } = route.params;
  const { service, loading, handleSubmitReview, handleStartConversation } =
    useServiceDetails(serviceId);
  const [currentImageIndex, setCurrentImageIndex] = useState(0);
  const [isReviewModalVisible, setReviewModalVisible] = useState(false);

  const imageWidth = Dimensions.get('window').width;

  if (loading) {
    return (
      <ActivityIndicator
        style={styles.loadingContainer}
        size="large"
        color="#3F83F8"
      />
    );
  }
  if (!service) {
    return (
      <View style={styles.loadingContainer}>
        <Text>Serviço não encontrado.</Text>
      </View>
    );
  }

  return (
    <SafeAreaView style={styles.safeArea}>
      <StatusBar barStyle="dark-content" />
      <ReviewModal
        visible={isReviewModalVisible}
        onClose={() => setReviewModalVisible(false)}
        onSubmit={async (rating, comment) => {
          await handleSubmitReview(rating, comment);
          setReviewModalVisible(false);
        }}
      />
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        {service.photo_urls && service.photo_urls.length > 0 && (

```



```

<FlatList
  data={service.photo_urls}
  renderItem={({ item }) => (
    <Image
      source={{ uri: item }}
      style={[styles.image, { width: imageWidth }]}
    />
  )}
  keyExtractor={item => item}
  horizontal
  pagingEnabled
  showsHorizontalScrollIndicator={false}
  onScroll={e => {
    const index = Math.round(
      e.nativeEvent.contentOffset.x / imageWidth,
    );
    setCurrentImageIndex(index);
  }}
/>
)}
{service.photo_urls && service.photo_urls.length > 1 && (
  <Text style={styles.imageCounter}>
    {currentImageIndex + 1} / {service.photo_urls.length}
  </Text>
)}
<View style={styles.contentContainer}>
  <Text style={styles.categoryTag}>
    {service.categories?.name || 'Sem Categoria'}
  </Text>
  <Text style={styles.title}>{service.title}</Text>
  <TouchableOpacity style={styles.providerContainer}>
    <Image
      source={{ uri: service.profiles?.avatar_url }}
      style={styles.providerAvatar}
    />
    <View>
      <Text>Oferecido por</Text>
      <Text style={styles.providerName}>
        {service.profiles?.full_name || 'Usuário Anônimo'}
      </Text>
    </View>
  </TouchableOpacity>
</View>
<View style={styles.contentContainer}>
  <Text style={styles.sectionTitle}>Detalhes do serviço</Text>
  <Text style={styles.description}>
    {service.description || 'Nenhuma descrição fornecida.'}
  </Text>

  <View style={styles.reviewsHeader}>
    <Text style={styles.sectionTitle}>Avaliações</Text>
    <TouchableOpacity
      style={styles.addReviewButton}
      onPress={() => setReviewModalVisible(true)}
    >

```

```

        >
        <Text style={styles.addReviewButtonText}>Avaliar</Text>
      </TouchableOpacity>
    </View>
    {service.reviews && service.reviews.length > 0 ? (
      service.reviews.map(review => (
        <ReviewItem key={review.id} review={review} />
      ))
    ) : (
      <Text style={styles.noReviewsText}>
        Seja o primeiro a avaliar este serviço!
      </Text>
    )}
  </View>
</ScrollView>
<View style={styles.footer}>
  <View>
    <Text style={styles.priceLabel}>A partir de</Text>
    <Text style={styles.priceValue}>
      R$ {service.price?.toFixed(2).replace('.', ',') || 'A
combinar'}
    </Text>
  </View>
  <TouchableOpacity
    style={styles.actionButton}
    onPress={handleStartConversation}
  >
    <Text style={styles.actionButtonText}>Pedir Orçamento</Text>
  </TouchableOpacity>
</View>
</SafeAreaView>
);
};

export default ServiceDetailScreen;

```

 ./src/screens/chat/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },
  keyboardAvoidingContainer: {
    flex: 1,
  },
  messageList: {
    flex: 1,
  },
});

```

```
paddingHorizontal: 10,
},
bubbleContainer: {
  maxWidth: '80%',
  marginBottom: 10,
},
myBubbleContainer: {
  alignSelf: 'flex-end',
},
recipientBubbleContainer: {
  alignSelf: 'flex-start',
},
messageBubble: {
  paddingHorizontal: 15,
  paddingVertical: 10,
  borderRadius: 20,
},
recipientBubble: {
  backgroundColor: '#EFEFEF',
},
myBubble: {
  backgroundColor: '#3F83F8',
},
messageText: {
  fontSize: 16,
},
myMessageText: {
  color: '#FFFFFF',
},
recipientMessageText: {
  color: '#000000',
},
timestamp: {
  fontSize: 12,
  color: '#999',
  marginTop: 4,
},
myTimestamp: {
  alignSelf: 'flex-end',
},
recipientTimestamp: {
  alignSelf: 'flex-start',
},
dateSeparator: {
  alignSelf: 'center',
  backgroundColor: '#E0E0E0',
  borderRadius: 12,
  paddingHorizontal: 12,
  paddingVertical: 6,
  marginVertical: 15,
},
dateSeparatorText: {
  color: '#666',
  fontWeight: 'bold',
```

```

    fontSize: 12,
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    paddingHorizontal: 10,
    paddingTop: 8,
    borderTopWidth: 1,
    borderTopColor: '#DDD',
    backgroundColor: '#FFFFFF',
  },
  input: {
    flex: 1,
    minHeight: 40,
    backgroundColor: '#F0F0F0',
    borderRadius: 20,
    paddingHorizontal: 15,
    marginRight: 10,
    fontSize: 16,
    paddingTop: 10,
  },
  sendButton: {
    backgroundColor: '#3F83F8',
    width: 40,
    height: 40,
    borderRadius: 20,
    justifyContent: 'center',
    alignItems: 'center',
  },
});

```

 ./src/screens/chat/index.tsx

```

import React, { useState, useEffect, useMemo, useCallback } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  FlatList,
  KeyboardAvoidingView,
  Platform,
  ActivityIndicator,
  Alert,
} from 'react-native';
import { useRoute, RouteProp } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { supabase } from '../../services/supabase';
import type { AppStackParamList } from '../../navigation/types';

```

```
import { styles } from './style';
import { format, isSameDay, isToday, isYesterday } from 'date-fns';
import { ptBR } from 'date-fns/locale';

type Message = {
  id: number;
  content: string;
  sender_id: string;
  created_at: string;
};

type ListItem =
  | { type: 'message'; data: Message }
  | { type: 'date'; date: string };

/**
 * Hook customizado para gerenciar o estado das mensagens de um chat.
 * Busca as mensagens iniciais e se inscreve para atualizações em tempo
 * real.
 * @param conversationId O ID da conversa para buscar mensagens.
 * @returns Um objeto contendo as mensagens, o estado de carregamento e o
 * ID do usuário atual.
 */
const useChatMessages = (conversationId: number) => {
  const [messages, setMessages] = useState<Message[]>([]);
  const [loading, setLoading] = useState(true);
  const [userId, setUserId] = useState<string | null>(null);

  useEffect(() => {
    const fetchUser = async () => {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      setUserId(user?.id || null);
    };
    fetchUser();
  }, []);

  useEffect(() => {
    if (!conversationId || !userId) return;

    const fetchMessages = async () => {
      const { data, error } = await supabase
        .from('messages')
        .select('*')
        .eq('conversation_id', conversationId)
        .order('created_at', { ascending: false });

      if (error) {
        Alert.alert('Erro', 'Não foi possível carregar as mensagens.');
```

```

};
fetchMessages();

const channel = supabase
  .channel(`chat_room_${conversationId}`)
  .on(
    'postgres_changes',
    {
      event: 'INSERT',
      schema: 'public',
      table: 'messages',
      filter: `conversation_id=eq.${conversationId}`,
    },
    payload => {
      if (payload.new.sender_id !== userId) {
        setMessages(prev => [payload.new as Message, ...prev]);
      }
    },
  )
  .subscribe();

return () => {
  supabase.removeChannel(channel);
};
}, [conversationId, userId]);

return { messages, setMessages, loading, userId };
};

const DateSeparator = ({ date }: { date: string }) => (
  <View style={styles.dateSeparator}>
    <Text style={styles.dateSeparatorText}>{date}</Text>
  </View>
);

const MessageBubble = ({
  message,
  isMyMessage,
}: {
  message: Message;
  isMyMessage: boolean;
}) => (
  <View
    style={[
      styles.bubbleContainer,
      isMyMessage ? styles.myBubbleContainer :
styles.recipientBubbleContainer,
    ]}
  >
    <View
      style={[
        styles.messageBubble,
        isMyMessage ? styles.myBubble : styles.recipientBubble,
      ]}

```

```

    >
    <Text
      style={isMyMessage ? styles.myMessageText :
styles.recipientMessageText}
    >
      {message.content}
    </Text>
  </View>
  <Text
    style={[
      styles.timestamp,
      isMyMessage ? styles.myTimestamp : styles.recipientTimestamp,
    ]}
  >
    {format(new Date(message.created_at), 'HH:mm')}
  </Text>
</View>
);

const ChatScreen = () => {
  const route = useRoute<RouteProp<AppStackParamList, 'Chat'>>();
  const { conversationId } = route.params;
  const insets = useSafeAreaInsets();

  const { messages, setMessages, loading, userId } =
    useChatMessages(conversationId);
  const [newMessage, setNewMessage] = useState('');

  const processedMessages = useMemo(() => {
    const items: ListItem[] = [];
    let lastDate: Date | null = null;

    messages.forEach(message => {
      const messageDate = new Date(message.created_at);
      if (!lastDate || !isSameDay(messageDate, lastDate)) {
        let dateString;
        if (isToday(messageDate)) {
          dateString = 'Hoje';
        } else if (isYesterday(messageDate)) {
          dateString = 'Ontem';
        } else {
          dateString = format(messageDate, "dd 'de' MMMM", { locale: ptBR
});
        }

        items.push({ type: 'date', date: dateString });
        lastDate = messageDate;
      }
      items.push({ type: 'message', data: message });
    });
    return items;
  }, [messages]);

  const handleSendMessage = async () => {

```

```

const content = newMessage.trim();
if (!content || !userId) return;

const optimisticMessage: Message = {
  id: Math.random(),
  content: content,
  sender_id: userId,
  created_at: new Date().toISOString(),
};

setMessages(prev => [optimisticMessage, ...prev]);
setNewMessage('');

const { error } = await supabase
  .from('messages')
  .insert({ conversation_id: conversationId, sender_id: userId, content
});

if (error) {
  Alert.alert('Erro', 'Não foi possível enviar a mensagem.');
```

```

  setMessages(prev => prev.filter(m => m.id !== optimisticMessage.id));
  setNewMessage(content);
}
};

const renderItem = useCallback(
  ({ item }: { item: ListItem }) => {
    if (item.type === 'date') {
      return <DateSeparator date={item.date} />;
    }
    return (
      <MessageBubble
        message={item.data}
        isMyMessage={item.data.sender_id === userId}
      />
    );
  },
  [userId],
);

if (loading)
  return <ActivityIndicator style={{ flex: 1, justifyContent: 'center' }} />;

return (
  <View style={[styles.container, { paddingTop: insets.top }]}>
    <KeyboardAvoidingView
      style={styles.keyboardAvoidingContainer}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
      keyboardVerticalOffset={90}
    >
      <FlatList
        data={processedMessages}
        keyExtractor={({ item, index }) =>

```



```

        item.type === 'message' ? item.data.id.toString() :
`date-${index}`
    }
    inverted
    style={styles.messageList}
    contentContainerStyle={{ paddingTop: 10 }}
    renderItem={renderItem}
  />
  <View
    style={[
      styles.inputContainer,
      { paddingBottom: insets.bottom > 0 ? insets.bottom : 8 },
    ]}
  >
    <TextInput
      style={styles.input}
      value={newMessage}
      onChangeText={setNewMessage}
      placeholder="Digite sua mensagem..."
      multiline
    />
    <TouchableOpacity
      style={styles.sendButton}
      onPress={handleSendMessage}
    >
      <Ionicons name="send" size={20} color="#FFFFFF" />
    </TouchableOpacity>
  </View>
</KeyboardAvoidingView>
</View>
);
};

export default ChatScreen;

```

📁 ./src/screens/profile/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#F9F9F9',
  },
  scrollContainer: {
    paddingBottom: 20,
  },
  profileHeader: {
    backgroundColor: '#FFFFFF',
    paddingVertical: 24,
  },

```

```
paddingHorizontal: 20,
alignItems: 'center',
borderBottomWidth: 1,
borderBottomColor: '#E2E8F0',
},
avatar: {
width: 90,
height: 90,
borderRadius: 45,
marginBottom: 12,
backgroundColor: '#E2E8F0',
},
avatarPlaceholder: {
justifyContent: 'center',
alignItems: 'center',
},
fullName: {
fontSize: 22,
fontWeight: 'bold',
color: '#1A202C',
},
editProfileButton: {
marginTop: 6,
},
editProfileText: {
fontSize: 16,
color: '#3F83F8',
fontWeight: '500',
},
sectionTitle: {
fontSize: 14,
fontWeight: 'bold',
color: '#718096',
textTransform: 'uppercase',
marginTop: 24,
marginBottom: 8,
paddingHorizontal: 20,
},
menuGroup: {
backgroundColor: '#FFFFFF',
borderTopWidth: 1,
borderBottomWidth: 1,
borderColor: '#E2E8F0',
},
menuItem: {
paddingHorizontal: 20,
paddingVertical: 16,
flexDirection: 'row',
alignItems: 'center',
},
menuItemBorder: {
borderBottomWidth: 1,
borderBottomColor: '#E2E8F0',
},
},
```

```
menuItemContent: {
  flex: 1,
  flexDirection: 'row',
  alignItems: 'center',
},
menuItemText: {
  fontSize: 17,
  marginLeft: 16,
  color: '#2D3748',
},
logoutButton: {
  margin: 20,
  backgroundColor: '#FFFFFF',
  paddingVertical: 16,
  borderRadius: 12,
  alignItems: 'center',
  borderWidth: 1.5,
  borderColor: '#E2E8F0',
},
logoutButtonText: {
  color: '#E53E3E',
  fontSize: 17,
  fontWeight: '600',
},
});
```

📁 ./src/screens/profile/index.tsx

```
import React, { useState, useCallback } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  Alert,
  SafeAreaView,
  ActivityIndicator,
  Image,
  ScrollView,
  StatusBar,
} from 'react-native';
import { useNavigation, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { supabase } from '../../../services/supabase';
import type { AppStackParamList } from '../../../navigation/types';
import { styles } from './style';
import { Ionicons } from '@expo/vector-icons';

type ProfileData = {
  fullName: string | null;
  avatarUrl: string | null;
```

```

};
type MenuItemProps = {
  icon: keyof typeof Ionicons.glyphMap;
  text: string;
  onPress: () => void;
  isLast?: boolean;
};
type ProfileScreenNavigationProp = NativeStackNavigationProp<
  AppStackParamList,
  'MainTabs'
>;

/**
 * Hook para buscar e gerenciar os dados do perfil do usuário.
 * Atualiza os dados sempre que a tela entra em foco.
 */
const useUserProfile = () => {
  const [profile, setProfile] = useState<ProfileData>({
    fullName: null,
    avatarUrl: null,
  });
  const [loading, setLoading] = useState(true);

  // useEffect com useCallback garante que a função de busca seja
  // recriada
  // apenas se necessário e executada toda vez que a tela focar.
  useEffect(
    useCallback(() => {
      const fetchProfile = async () => {
        try {
          setLoading(true);
          const {
            data: { user },
          } = await supabase.auth.getUser();
          if (!user) throw new Error('Usuário não autenticado');

          const { data, error } = await supabase
            .from('profiles')
            .select(`full_name, avatar_url`)
            .eq('id', user.id)
            .single();

          if (error && error.code !== 'PGRST116') throw error;

          if (data) {
            setProfile({
              fullName: data.full_name,
              avatarUrl: data.avatar_url
                ? `${data.avatar_url}?t=${new Date().getTime()}`
                : null,
            });
          }
        } catch (error) {
          const message =

```

```

        error instanceof Error
        ? error.message
        : 'Ocorreu um erro desconhecido.';
        Alert.alert('Erro ao carregar perfil', message);
      } finally {
        setLoading(false);
      }
    };
    fetchProfile();
  }, []),
);

return { profile, loading };
};

/**
 * Subcomponente reutilizável para renderizar um item do menu do perfil.
 */
const MenuItem: React.FC<MenuItemProps> = ({
  icon,
  text,
  onPress,
  isLast = false,
}) => (
  <TouchableOpacity
    style={[styles.menuItem, !isLast && styles.menuItemBorder]}
    onPress={onPress}
  >
    <View style={styles.menuItemContent}>
      <Ionicons name={icon} size={24} color="#718096" />
      <Text style={styles.menuItemText}>{text}</Text>
    </View>
    <Ionicons name="chevron-forward-outline" size={24} color="#CBD5E0" />
  </TouchableOpacity>
);

/**
 * Tela de Perfil do usuário. Exibe um resumo das informações e serve como
 * ponto de entrada para o gerenciamento da conta e outras seções do app.
 */
const ProfileScreen: React.FC = () => {
  const navigation = useNavigation<ProfileScreenNavigationProp>();
  const { profile, loading } = useUserProfile();

  /**
   * Lida com o logout do usuário, exibindo um `Alert` de confirmação
   antes.
   */
  const handleSignOut = () => {
    Alert.alert('Sair da Conta', 'Você tem certeza que deseja sair?', [
      { text: 'Cancelar', style: 'cancel' },
      {
        text: 'Sair',
        style: 'destructive',
      },
    ],
  );
};

```

```

        onPress: () => supabase.auth.signOut(),
      },
    ]);
  };

  if (loading) {
    return (
      <SafeAreaView style={styles.safeArea}>
        <ActivityIndicator style={{ flex: 1 }} size="large" />
      </SafeAreaView>
    );
  }

  return (
    <SafeAreaView style={styles.safeArea}>
      <StatusBar barStyle="dark-content" />
      <ScrollView style={styles.scrollContainer}>
        <View style={styles.profileHeader}>
          {profile.avatarUrl ? (
            <Image source={{ uri: profile.avatarUrl }} style=
{styles.avatar} />
          ) : (
            <View style={[styles.avatar, styles.avatarPlaceholder]}>
              <Ionicons name="person" size={50} color="#CBD5E0" />
            </View>
          )}
          <Text style={styles.fullName}>
            {profile.fullName || 'Complete seu Perfil'}
          </Text>
          <TouchableOpacity
            style={styles.editProfileButton}
            onPress={() => navigation.navigate('EditProfile')}
          >
            <Text style={styles.editProfileText}>Editar perfil</Text>
          </TouchableOpacity>
        </View>

        <Text style={styles.sectionTitle}>Minha Conta</Text>
        <View style={styles.menuGroup}>
          <MenuItem
            icon="key-outline"
            text="Alterar senha"
            onPress={() => navigation.navigate('ChangePassword')}
          />
          <MenuItem
            icon="receipt-outline"
            text="Meus Serviços"
            onPress={() =>
              Alert.alert(
                'Em breve!',
                'Aqui você poderá ver e gerenciar os serviços que
cadastrou.',
              )
            }
          />
        </View>
      </SafeAreaView>
    );
  }

```

```

        isLast={true}
      />
    </View>

    <Text style={styles.sectionTitle}>Ajuda</Text>
    <View style={styles.menuGroup}>
      <MenuItem
        icon="help-circle-outline"
        text="Central de Ajuda"
        onPress={() =>
          Alert.alert('Em breve!', 'Funcionalidade em
desenvolvimento.')
        }
      />
      <MenuItem
        icon="document-text-outline"
        text="Termos de uso"
        onPress={() =>
          Alert.alert('Em breve!', 'Funcionalidade em
desenvolvimento.')
        }
        isLast={true}
      />
    </View>

    <TouchableOpacity style={styles.logoutButton} onPress=
{handleSignOut}>
      <Text style={styles.logoutButtonText}>Sair</Text>
    </TouchableOpacity>
  </ScrollView>
</SafeAreaView>
);
};

export default ProfileScreen;

```

 ./src/screens/search/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },
  header: {
    padding: 15,
    backgroundColor: '#FFFFFF',
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
  },
});

```

```
},
inputContainer: {
  flexDirection: 'row',
  alignItems: 'center',
  backgroundColor: '#F7FAFC',
  borderRadius: 12,
  paddingHorizontal: 15,
},
searchInput: {
  flex: 1,
  height: 48,
  fontSize: 16,
  marginLeft: 10,
  color: '#2D3748',
},
filtersContainer: {
  flexDirection: 'row',
  marginTop: 12,
  gap: 10,
},
filterChip: {
  flexDirection: 'row',
  alignItems: 'center',
  backgroundColor: '#E6F0FF',
  borderRadius: 20,
  paddingVertical: 8,
  paddingHorizontal: 15,
},
filterChipText: {
  color: '#3F83F8',
  fontWeight: '600',
  fontSize: 14,
  marginLeft: 6,
},
listContentContainer: {
  padding: 20,
},
emptyContainer: {
  flex: 1,
  justifyContent: 'center',
  alignItems: 'center',
  padding: 20,
  marginTop: 50,
},
emptyIcon: {
  marginBottom: 20,
},
emptyTitle: {
  fontSize: 18,
  fontWeight: 'bold',
  color: '#2D3748',
},
emptySubtitle: {
  fontSize: 14,
```



```

    color: '#718096',
    textAlign: 'center',
    marginTop: 8,
  },
  modalOverlay: {
    flex: 1,
    justifyContent: 'flex-end',
    backgroundColor: 'rgba(0,0,0,0.5)',
  },
  modalContent: {
    backgroundColor: 'white',
    borderTopLeftRadius: 20,
    borderTopRightRadius: 20,
    padding: 20,
    maxHeight: '80%',
  },
  modalHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 20,
  },
  modalTitle: { fontSize: 20, fontWeight: 'bold' },
  categoryItem: {
    flexDirection: 'row',
    alignItems: 'center',
    paddingVertical: 15,
    borderBottomWidth: 1,
    borderBottomColor: '#EEE',
  },
  categoryText: { fontSize: 18, marginLeft: 15 },
  modalFooter: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 20,
  },
  modalButton: { flex: 1, padding: 15, borderRadius: 8, alignItems:
'center' },
  closeButton: { backgroundColor: '#EEE', marginRight: 10 },
  saveButton: { backgroundColor: '#3F83F8', marginLeft: 10 },
  modalButtonText: { fontSize: 16, fontWeight: 'bold' },
});

```

📁 ./src/screens/search/index.tsx

```

import React, { useState, useCallback, useEffect } from 'react';
import {
  View,
  Text,
  FlatList,

```

```
Alert,
ActivityIndicator,
TextInput,
TouchableOpacity,
Modal,
StatusBar,
} from 'react-native';
import { supabase } from '../../services/supabase';
import { styles } from './style';
import { Ionicons } from '@expo/vector-icons';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import ServiceCard, { ServiceCardData } from '../../components/ServiceCard';
import useDebounce from '../../hooks/useDebounce';

// --- Tipos ---
type Category = { id: number; name: string };

/**
 * Hook para gerenciar a busca, filtros e carregamento de serviços.
 */
const useServiceSearch = () => {
  const [services, setServices] = useState<ServiceCardData[]>([]);
  const [loading, setLoading] = useState(true);
  const [searchTerm, setSearchTerm] = useState('');
  const [selectedCategories, setSelectedCategories] = useState<number[]>([]);
  const debouncedSearchTerm = useDebounce(searchTerm, 500);

  const fetchServices = useCallback(async () => {
    setLoading(true);
    try {
      const { data, error } = await
supabase.rpc('get_services_with_ratings', {
        search_term:
          debouncedSearchTerm.trim() === '' ? null : debouncedSearchTerm,
        category_ids_filter:
          selectedCategories.length === 0 ? null : selectedCategories,
      });

      if (error) throw error;
      setServices(data || []);
    } catch (error) {
      const message =
        error instanceof Error
          ? error.message
          : 'Ocorreu um erro desconhecido.';
      Alert.alert('Erro ao buscar serviços', message);
    } finally {
      setLoading(false);
    }
  }, [debouncedSearchTerm, selectedCategories]);
```

```
useEffect(() => {
  fetchServices();
}, [fetchServices]);

return {
  services,
  loading,
  searchTerm,
  setSearchTerm,
  selectedCategories,
  setSelectedCategories,
};
};

// --- Subcomponente para o Modal de Filtro ---
type FilterModalProps = {
  visible: boolean;
  onClose: () => void;
  onApply: (selectedIds: number[]) => void;
  initialSelectedCategories: number[];
};

const FilterModal: React.FC<FilterModalProps> = ({
  visible,
  onClose,
  onApply,
  initialSelectedCategories,
}) => {
  const [availableCategories, setAvailableCategories] =
    useState<Category[]>(
      [],
    );
  const [tempSelected, setTempSelected] = useState<number[]>(
    initialSelectedCategories,
  );

  useEffect(() => {
    // Busca categorias apenas uma vez quando o modal é preparado para
    abrir
    if (visible && availableCategories.length === 0) {
      supabase
        .from('categories')
        .select('id, name')
        .then(({ data }) => {
          setAvailableCategories(data || []);
        });
    }
    // Sincroniza o estado temporário com o estado real quando o modal abre
    setTempSelected(initialSelectedCategories);
  }, [visible, initialSelectedCategories, availableCategories.length]);

  const handleToggleCategory = (id: number) => {
    setTempSelected(prev =>
      prev.includes(id) ? prev.filter(catId => catId !== id) : [...prev,
```

```

id],
  );
};

return (
  <Modal
    animationType="slide"
    transparent={true}
    visible={visible}
    onRequestClose={onClose}
  >
    <View style={styles.modalOverlay}>
      <View style={styles.modalContent}>
        <View style={styles.modalHeader}>
          <Text style={styles.modalTitle}>Filtrar categorias</Text>
          <TouchableOpacity onPress={onClose}>
            <Ionicons name="close-circle" size={30} color="#CCC" />
          </TouchableOpacity>
        </View>
        <FlatList
          data={availableCategories}
          keyExtractor={item => item.id.toString()}
          renderItem={({ item }) => {
            const isSelected = tempSelected.includes(item.id);
            return (
              <TouchableOpacity
                style={styles.categoryItem}
                onPress={() => handleToggleCategory(item.id)}
              >
                <Ionicons
                  name={isSelected ? 'checkbox' : 'square-outline'}
                  size={24}
                  color={isSelected ? '#3F83F8' : '#CCC'}
                />
                <Text style={styles.categoryText}>{item.name}</Text>
              </TouchableOpacity>
            );
          }}
        />
        <View style={styles.modalFooter}>
          <TouchableOpacity
            style={[styles.modalButton, styles.closeButton]}
            onPress={onClose}
          >
            <Text style={styles.modalButtonText}>Fechar</Text>
          </TouchableOpacity>
          <TouchableOpacity
            style={[styles.modalButton, styles.saveButton]}
            onPress={() => onApply(tempSelected)}
          >
            <Text style={[styles.modalButtonText, { color: '#FFF' }]}>
              Salvar
            </Text>
          </TouchableOpacity>

```

```
        </View>
      </View>
    </View>
  </Modal>
);
};

/**
 * Tela de busca de serviços. Permite busca por texto e filtro por
 * categorias.
 */
const SearchScreen: React.FC = () => {
  const {
    services,
    loading,
    searchTerm,
    setSearchTerm,
    selectedCategories,
    setSelectedCategories,
  } = useServiceSearch();
  const [isModalVisible, setModalVisible] = useState(false);

  const handleApplyFilters = (categories: number[]) => {
    setSelectedCategories(categories);
    setModalVisible(false);
  };

  const renderEmptyList = () => (
    <View style={styles.emptyContainer}>
      <Ionicons
        name="search-circle-outline"
        size={60}
        color="#CBD5E0"
        style={styles.emptyIcon}
      />
      <Text style={styles.emptyTitle}>Nenhum serviço encontrado</Text>
      <Text style={styles.emptySubtitle}>
        Tente ajustar sua busca ou filtros para encontrar o que procura.
      </Text>
    </View>
  );

  return (
    <SafeAreaView>
      <SafeAreaView style={styles.safeArea}>
        <StatusBar barStyle="dark-content" />

        <FilterModal
          visible={isModalVisible}
          onClose={() => setModalVisible(false)}
          onApply={handleApplyFilters}
          initialSelectedCategories={selectedCategories}
        />
      </SafeAreaView>
    </SafeAreaView>
  );
};
```

```

<View style={styles.header}>
  <View style={styles.inputContainer}>
    <Ionicons name="search" size={20} color="#999" />
    <TextInput
      style={styles.searchInput}
      placeholder="Buscar por serviço ou especialidade..."
      value={searchTerm}
      onChangeText={setSearchTerm}
      returnKeyType="search"
    />
  </View>
  <View style={styles.filtersContainer}>
    <TouchableOpacity
      style={styles.filterChip}
      onPress={() => setModalVisible(true)}
    >
      <Ionicons name="options-outline" size={16} color="#3F83F8" />
      <Text style={styles.filterChipText}>
        {selectedCategories.length > 0
          ? ` ${selectedCategories.length} Categoria(s)`
          : 'Filtrar'}
      </Text>
    </TouchableOpacity>
  </View>
</View>

{loading ? (
  <ActivityIndicator style={{ flex: 1 }} size="large"
color="#3F83F8" />
) : (
  <FlatList
    data={services}
    renderItem={({ item }) => <ServiceCard service={item} />}
    keyExtractor={item => item.id.toString()}
    contentContainerStyle={styles.listContentContainer}
    ListEmptyComponent={renderEmptyList}
  />
)}
</SafeAreaView>
</SafeAreaProvider>
);
};

export default SearchScreen;

```

📁 ./src/screens/editProfile/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({

```

```
container: {
  flex: 1,
  padding: 20,
  backgroundColor: '#F9F9F9',
},
label: {
  fontSize: 16,
  color: '#666',
  marginBottom: 5,
  marginTop: 15,
},
input: {
  height: 50,
  backgroundColor: '#FFFFFF',
  borderColor: '#CCCCCC',
  borderWidth: 1,
  borderRadius: 8,
  paddingHorizontal: 15,
  fontSize: 16,
  color: '#333',
},
button: {
  backgroundColor: '#3F83F8',
  paddingVertical: 15,
  borderRadius: 8,
  alignItems: 'center',
  marginTop: 30,
},
buttonText: {
  color: '#FFFFFF',
  fontSize: 18,
  fontWeight: 'bold',
},
avatarContainer: {
  alignItems: 'center',
  marginBottom: 20,
  marginTop: 10,
},
avatar: {
  width: 120,
  height: 120,
  borderRadius: 60,
  backgroundColor: '#EFEFEF',
  justifyContent: 'center',
  alignItems: 'center',
},
avatarPlaceholder: {
  width: 120,
  height: 120,
  borderRadius: 60,
  backgroundColor: '#EFEFEF',
  justifyContent: 'center',
  alignItems: 'center',
},
```

```
editAvatarButton: {
  position: 'absolute',
  bottom: 0,
  right: 120,
  backgroundColor: '#3F83F8',
  borderRadius: 20,
  padding: 8,
  borderWidth: 2,
  borderColor: '#FFF',
},
locationButton: {
  flexDirection: 'row',
  alignItems: 'center',
  height: 50,
  backgroundColor: 'FFFFFF',
  borderColor: 'CCCCCC',
  borderWidth: 1,
  borderRadius: 8,
  paddingHorizontal: 15,
},
locationText: {
  flex: 1,
  fontSize: 16,
  color: '#333',
},
locationPlaceholder: {
  flex: 1,
  fontSize: 16,
  color: '#999',
},
});
```

📁 ./src/screens/editProfile/index.tsx

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  Alert,
  SafeAreaView,
  ActivityIndicator,
  Image,
  ScrollView,
} from 'react-native';
import { supabase } from '../../../services/supabase';
import { styles } from './style';
import * as ImagePicker from 'expo-image-picker';
import { Ionicons } from '@expo/vector-icons';
```



```
import { decode } from 'base64-arraybuffer';
import LocationSelectorModal from '../components/LocationSelectorModal';

type Profile = {
  username: string;
  full_name: string;
  phone: string;
  avatar_url: string | null;
  location: string;
};

/**
 * Hook para gerenciar os dados do perfil do usuário.
 * Encapsula a lógica de buscar, atualizar o perfil e o avatar.
 */
const useProfile = () => {
  const [profile, setProfile] = useState<Partial<Profile>>({});
  const [loading, setLoading] = useState(true);
  const [isDirty, setIsDirty] = useState(false);

  useEffect(() => {
    const fetchProfile = async () => {
      setLoading(true);
      try {
        const {
          data: { user },
        } = await supabase.auth.getUser();
        if (!user) throw new Error('Usuário não encontrado');

        const { data, error } = await supabase
          .from('profiles')
          .select(`username, full_name, phone, avatar_url, location`)
          .eq('id', user.id)
          .single();

        if (error && error.code !== 'PGRST116') throw error;
        if (data) setProfile(data);
      } catch (error) {
        const message =
          error instanceof Error
            ? error.message
            : 'Ocorreu um erro desconhecido.';
        Alert.alert('Erro ao carregar dados', message);
      } finally {
        setLoading(false);
      }
    };
  }, []);

  /** Atualiza um campo específico do perfil */
  const updateProfileField = (field: keyof Profile, value: string | null)
  => {
    setProfile(prev => ({ ...prev, [field]: value }));
  }
}
```

```
    setIsDirty(true);
  };

  /** Salva todas as mudanças do perfil no banco de dados */
  const saveProfileChanges = async () => {
    if (!isDirty) {
      Alert.alert('Nenhuma alteração', 'Não há nenhuma alteração para salvar.');
```

salvar.');

```
      return;
    }
    setLoading(true);
    try {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      if (!user) throw new Error('Usuário não encontrado');
```

const updates = { ...profile, updated_at: new Date() };

```
      const { error } = await supabase
        .from('profiles')
        .update(updates)
        .eq('id', user.id);
      if (error) throw error;
```

Alert.alert('Sucesso!', 'Seu perfil foi atualizado.');

```
      setIsDirty(false); // Reseta o estado de mudanças
    } catch (error) {
      const message =
        error instanceof Error
          ? error.message
          : 'Ocorreu um erro desconhecido.';
      Alert.alert('Erro ao atualizar', message);
    } finally {
      setLoading(false);
    }
  };

  /** Faz o upload de um novo avatar */
  const uploadAvatar = async (base64Image: string) => {
    setLoading(true);
    try {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      if (!user) throw new Error('Usuário não encontrado');
```

const filePath = `\${user.id}/\${new Date().getTime()}.png`;

```
      const { error: uploadError } = await supabase.storage
        .from('avatars')
        .upload(filePath, decode(base64Image), {
          contentType: 'image/png',
          upsert: true,
        });
```

```
    if (uploadError) throw uploadError;

    const { data: publicUrlData } = supabase.storage
      .from('avatars')
      .getPublicUrl(filePath);
    const newAvatarUrl = `${publicUrlData.publicUrl}?t=${new
Date().getTime()}`;

    await supabase
      .from('profiles')
      .update({ avatar_url: newAvatarUrl, updated_at: new Date() })
      .eq('id', user.id);
    updateProfileField('avatar_url', newAvatarUrl);

    Alert.alert('Sucesso!', 'Foto de perfil atualizada.');
```

```
} catch (error) {
  const message =
    error instanceof Error
      ? error.message
      : 'Ocorreu um erro desconhecido.';
  Alert.alert('Erro no upload', message);
} finally {
  setLoading(false);
}
};

return {
  profile,
  loading,
  updateProfileField,
  saveProfileChanges,
  uploadAvatar,
};
};

const EditProfileScreen = () => {
  const {
    profile,
    loading,
    updateProfileField,
    saveProfileChanges,
    uploadAvatar,
  } = useProfile();
  const [isLocationModalVisible, setLocationModalVisible] =
    useState(false);

  const handlePickImage = async () => {
    Alert.alert('Alterar Foto', 'Escolha uma opção', [
      { text: 'Tirar Foto', onPress: () => selectImage('camera') },
      { text: 'Escolher da Galeria', onPress: () => selectImage('gallery')
},
      { text: 'Cancelar', style: 'cancel' },
    ]);
  };
};
```

```
const selectImage = async (source: 'camera' | 'gallery') => {
  const options: ImagePicker.ImagePickerOptions = {
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [1, 1],
    quality: 0.7,
    base64: true,
  };

  let result;
  if (source === 'camera') {
    await ImagePicker.requestCameraPermissionsAsync();
    result = await ImagePicker.launchCameraAsync(options);
  } else {
    await ImagePicker.requestMediaLibraryPermissionsAsync();
    result = await ImagePicker.launchImageLibraryAsync(options);
  }

  if (!result.canceled && result.assets && result.assets[0].base64) {
    uploadAvatar(result.assets[0].base64);
  }
};

if (loading && !profile.avatar_url) {
  return (
    <ActivityIndicator
      style={{ flex: 1, justifyContent: 'center' }}
      size="large"
    />
  );
}

return (
  <SafeAreaView style={{ flex: 1, backgroundColor: '#F9F9F9' }}>
    <LocationSelectorModal
      visible={isLocationModalVisible}
      onClose={() => setLocationModalVisible(false)}
      onLocationSelect={selectedLocation => {
        updateProfileField('location', selectedLocation);
        setLocationModalVisible(false); // Fecha o modal após a seleção
      }}
    />
    <ScrollView style={styles.container}>
      <View style={styles.avatarContainer}>
        <Image
          source={{ uri: profile.avatar_url || undefined }}
          style={styles.avatar}
        />
        <TouchableOpacity
          onPress={handlePickImage}
          style={styles.editAvatarButton}
        >
          <Ionicons name="camera" size={20} color="#FFF" />
        </TouchableOpacity>
      </View>
    </ScrollView>
  </SafeAreaView>
);
```

```
    </TouchableOpacity>
  </View>

  <Text style={styles.label}>Nome de Usuário</Text>
  <TextInput
    style={styles.input}
    value={profile.username}
    onChangeText={text => updateProfileField('username', text)}
  />

  <Text style={styles.label}>Nome e Sobrenome</Text>
  <TextInput
    style={styles.input}
    value={profile.full_name}
    onChangeText={text => updateProfileField('full_name', text)}
  />

  <Text style={styles.label}>Telefone</Text>
  <TextInput
    style={styles.input}
    value={profile.phone}
    onChangeText={text => updateProfileField('phone', text)}
    keyboardType="phone-pad"
  />

  <Text style={styles.label}>Minha Localização</Text>
  <TouchableOpacity
    style={styles.locationButton}
    onPress={() => setLocationModalVisible(true)}
  >
    <Text
      style={
        profile.location
          ? styles.locationText
          : styles.locationPlaceholder
      }
    />
    {profile.location || 'Clique para selecionar sua cidade'}
  </Text>
  <Ionicons name="chevron-down" size={20} color="#666" />
</TouchableOpacity>

  <TouchableOpacity
    style={styles.button}
    onPress={saveProfileChanges}
    disabled={loading}
  >
    {loading ? (
      <ActivityIndicator color="#FFF" />
    ) : (
      <Text style={styles.buttonText}>Salvar Alterações</Text>
    )}
  </TouchableOpacity>
</ScrollView>
```

```
    </SafeAreaView>
  );
};

export default EditProfileScreen;
```

📁 ./src/screens/confirmEmail/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },
  keyboardAvoidingContainer: {
    flex: 1,
  },
  contentContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  icon: {
    marginBottom: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 15,
    color: '#333',
  },
  message: {
    fontSize: 16,
    textAlign: 'center',
    color: '#666',
    marginBottom: 30,
    lineHeight: 24,
  },
  otpInput: {
    width: '90%',
    height: 50,
    backgroundColor: '#F7FAFC',
    borderWidth: 1,
    borderColor: '#E2E8F0',
    borderRadius: 8,
    textAlign: 'center',
    fontSize: 22,
```

```
    letterSpacing: 10,
    marginBottom: 20,
    color: '#2D3748',
  },
  button: {
    width: '90%',
    backgroundColor: '#3F83F8',
    paddingVertical: 15,
    borderRadius: 8,
    alignItems: 'center',
  },
  buttonText: {
    color: '#FFFFFF',
    fontSize: 18,
    fontWeight: 'bold',
  },
  errorText: {
    color: '#E53E3E',
    marginBottom: 15,
    fontSize: 14,
    textAlign: 'center',
  },
  resendContainer: {
    marginTop: 30,
    alignItems: 'center',
  },
  resendText: {
    fontSize: 14,
    color: '#718096',
  },
  resendButton: {
    marginTop: 5,
  },
  resendButtonText: {
    fontSize: 14,
    color: '#3F83F8',
    fontWeight: 'bold',
  },
  resendStatusText: {
    fontSize: 14,
    color: '#48BB78',
    marginTop: 10,
  },
},
});
```

📁 ./src/screens/confirmEmail/index.tsx

```
import React, { useState, useEffect } from 'react';
import {
  View,
```

```
Text,
SafeAreaView,
TextInput,
TouchableOpacity,
ActivityIndicator,
KeyboardAvoidingView,
Platform,
StatusBar,
} from 'react-native';
import { useRoute, RouteProp } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { supabase } from '../../services/supabase';
import { styles } from './style';
import type { AuthStackParamList } from '../../navigation/types';

type ConfirmEmailRouteProp = RouteProp<AuthStackParamList, 'ConfirmEmail'>;
const COOLDOWN_SECONDS = 60;

/**
 * Hook customizado para gerenciar um contador regressivo.
 * @param initialCount O número inicial de segundos para a contagem.
 * @returns Um objeto com o tempo restante e uma função para reiniciar o
contador.
 */
const useCountdown = (initialCount: number) => {
  const [countdown, setCountdown] = useState(initialCount);
  const isRunning = countdown > 0;

  useEffect(() => {
    if (!isRunning) return;

    const timerId = setInterval(() => {
      setCountdown(current => current - 1);
    }, 1000);

    return () => clearInterval(timerId);
  }, [isRunning]);

  const resetCountdown = () => setCountdown(initialCount);

  return { countdown, isRunning, resetCountdown };
};

/**
 * Tela para o usuário inserir o código de verificação (OTP) enviado para
seu e-mail.
 */
const ConfirmEmailScreen: React.FC = () => {
  const route = useRoute<ConfirmEmailRouteProp>();
  const { email } = route.params;

  const [otp, setOtp] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
```



```
const [resendStatus, setResendStatus] = useState('');
const { countdown, isRunning, resetCountdown } =
  useCountdown(COOLDOWN_SECONDS);

/**
 * Valida e envia o código OTP para o Supabase para verificação.
 */
const handleConfirmOtp = async () => {
  setError('');
  if (otp.length !== 6) {
    setError('O código deve ter 6 dígitos.');
```

return;

setLoading(true);

```
const { error: verifyError } = await supabase.auth.verifyOtp({
  email: email,
  token: otp,
  type: 'signup',
});

if (verifyError) {
  setError('Código inválido ou expirado. Tente novamente.');
```

setLoading(false);

```
};

/**
 * Solicita o reenvio de um novo código OTP para o e-mail do usuário.
 */
const handleResendOtp = async () => {
  if (isRunning || loading) return;

  setLoading(true);
  setResendStatus('');
  setError('');
```

const { error: resendError } = await supabase.auth.resend({

type: 'signup',

email: email,

```
});

if (resendError) {
  setError('Ocorreu um erro ao reenviar o código.');
```

else {

setResendStatus('Um novo código foi enviado para seu e-mail.');

resetCountdown(); // Reinicia o contador

setLoading(false);

```
};

return (
  <SafeAreaView style={styles.safeArea}>
    <StatusBar barStyle="dark-content" />
```

```

<KeyboardAvoidingView
  style={styles.keyboardAvoidingContainer}
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
>
  <View style={styles.contentContainer}>
    <Ionicons
      name="mail-open-outline"
      size={80}
      color="#3F83F8"
      style={styles.icon}
    />
    <Text style={styles.title}>Verifique seu E-mail</Text>
    <Text style={styles.message}>
      Enviamos um código de 6 dígitos para {email}. Por favor,
      insira-o
      abaixo.
    </Text>
    <TextInput
      style={styles.otpInput}
      value={otp}
      onChangeText={setOtp}
      keyboardType="number-pad"
      maxLength={6}
      placeholder="-----"
      placeholderTextColor="#CBD5E0"
    />

    {error ? <Text style={styles.errorText}>{error}</Text> : null}

    <TouchableOpacity
      style={styles.button}
      onPress={handleConfirmOtp}
      disabled={loading}
    >
      {loading ? (
        <ActivityIndicator color="#FFFFFF" />
      ) : (
        <Text style={styles.buttonText}>Confirmar e Continuar</Text>
      )}
    </TouchableOpacity>

    <View style={styles.resendContainer}>
      {!isRunning ? (
        <TouchableOpacity
          style={styles.resendButton}
          onPress={handleResendOtp}
          disabled={loading}
        >
          <Text style={styles.resendButtonText}>
            Não recebeu? Reenviar código
          </Text>
        </TouchableOpacity>
      ) : (
        <Text style={styles.resendText}>

```

```

        Você pode reenviar o código em {countdown} segundos
      </Text>
    )}
    {resendStatus ? (
      <Text style={styles.resendStatusText}>{resendStatus}</Text>
    ) : null}
  </View>
</View>
</KeyboardAvoidingView>
</SafeAreaView>
);
};

export default ConfirmEmailScreen;

```

📁 ./src/screens/login/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },
  scrollContainer: {
    flexGrow: 1,
    justifyContent: 'center',
    padding: 20,
  },
  headerContainer: {
    alignItems: 'center',
    marginBottom: 40,
  },
  title: {
    fontSize: 32,
    fontWeight: 'bold',
    color: '#2D3748',
    marginBottom: 8,
  },
  subtitle: {
    fontSize: 18,
    color: '#718096',
  },
  formContainer: {
    width: '100%',
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#F7FAFC',

```

```
    borderWidth: 1,
    borderColor: '#E2E8F0',
    borderRadius: 8,
    marginBottom: 15,
    paddingHorizontal: 15,
  },
  input: {
    flex: 1,
    height: 50,
    fontSize: 16,
    color: '#2D3748',
  },
  icon: {
    marginLeft: 10,
  },
  forgotPasswordButton: {
    alignSelf: 'flex-end',
    marginBottom: 20,
  },
  forgotPasswordText: {
    color: '#3F83F8',
    fontSize: 14,
    fontWeight: '500',
  },
  footerContainer: {
    width: '100%',
    alignItems: 'center',
  },
  loginButton: {
    width: '100%',
    backgroundColor: '#3F83F8',
    paddingVertical: 16,
    borderRadius: 12,
    alignItems: 'center',
    elevation: 2,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.2,
    shadowRadius: 2,
  },
  loginButtonText: {
    color: '#FFFFFF',
    fontSize: 18,
    fontWeight: '600',
  },
  signupButton: {
    marginTop: 20,
  },
  signupText: {
    fontSize: 16,
    color: '#718096',
  },
  signupLink: {
    color: '#3F83F8',
```

```
    fontWeight: 'bold',
  },
  errorText: {
    color: '#E53E3E',
    textAlign: 'center',
    marginBottom: 10,
    fontSize: 14,
  },
  modalContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: 'rgba(0, 0, 0, 0.5)',
  },
  modalView: {
    width: 150,
    height: 150,
    backgroundColor: 'white',
    borderRadius: 20,
    justifyContent: 'center',
    alignItems: 'center',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.25,
    shadowRadius: 4,
    elevation: 5,
  },
});
```

📁 ./src/screens/login/index.tsx

```
import React, { useState, useCallback } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  SafeAreaView,
  ActivityIndicator,
  ScrollView,
  KeyboardAvoidingView,
  Platform,
  StatusBar,
} from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { Ionicons } from '@expo/vector-icons';
import { supabase } from '../../../services/supabase';
import { styles } from './style';
import type { AuthStackParamList } from '../../../navigation/types';
```

```
type LoginScreenNavigationProp = NativeStackNavigationProp<
  AuthStackParamList,
  'Login'
>;

/**
 * Tela de Login. Permite que usuários existentes acessem suas contas.
 */
const LoginScreen: React.FC = () => {
  const navigation = useNavigation<LoginScreenNavigationProp>();

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isPasswordVisible, setIsPasswordVisible] = useState(false);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  /**
   * Tenta autenticar o usuário com o e-mail e senha fornecidos.
   */
  const handleLogin = async () => {
    if (!email.trim() || !password) {
      setError('Por favor, preencha e-mail e senha.');
```

return;

```
    }

    setLoading(true);
    setError('');

    const { error: signInError } = await supabase.auth.signInWithPassword({
      email: email.trim(),
      password: password,
    });

    if (signInError) {
      if (signInError.message === 'Email not confirmed') {
        setError('Por favor, confirme seu e-mail antes de fazer login.');
```

} else {

```
        setError('E-mail ou senha inválidos.');
```

}

```
    }
    setLoading(false);
  };

  /**
   * Limpa o erro ao digitar no campo de e-mail.
   */
  const handleEmailChange = useCallback(
    (text: string) => {
      setEmail(text);
      if (error) setError('');
    },
    [error],
  );
}
```

```
);

/**
 * Limpa o erro ao digitar no campo de senha.
 */
const handlePasswordChange = useCallback(
  (text: string) => {
    setPassword(text);
    if (error) setError('');
  },
  [error],
);

return (
  <SafeAreaView style={styles.safeArea}>
    <StatusBar barStyle="dark-content" />
    <KeyboardAvoidingView
      style={{ flex: 1 }}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    >
      <ScrollView
        contentContainerStyle={styles.scrollContainer}
        keyboardShouldPersistTaps="handled"
      >
        <View style={styles.headerContainer}>
          <Text style={styles.title}>Bem-vindo(a) de volta!</Text>
          <Text style={styles.subtitle}>
            Acesse sua conta para continuar.
          </Text>
        </View>

        <View style={styles.formContainer}>
          <View style={styles.inputContainer}>
            <TextInput
              style={styles.input}
              placeholder="E-mail"
              value={email}
              onChangeText={handleEmailChange}
              keyboardType="email-address"
              autoCapitalize="none"
            />
          </View>

          <View style={styles.inputContainer}>
            <TextInput
              style={styles.input}
              placeholder="Senha"
              value={password}
              onChangeText={handlePasswordChange}
              secureTextEntry={!isPasswordVisible}
            />
            <TouchableOpacity
              onPress={() => setIsPasswordVisible(prev => !prev)}
            >
```

```

        <Ionicons
          name={isPasswordVisible ? 'eye-off' : 'eye'}
          size={24}
          color="#A0AEC0"
          style={styles.icon}
        />
      </TouchableOpacity>
    </View>

    {error ? <Text style={styles.errorText}>{error}</Text> : null}

    <TouchableOpacity
      style={styles.forgotPasswordButton}
      onPress={() => navigation.navigate('ForgotPassword')}
    >
      <Text style={styles.forgotPasswordText}>Esqueceu a senha?
    </Text>
    </TouchableOpacity>
  </View>

  <View style={styles.footerContainer}>
    <TouchableOpacity
      style={styles.loginButton}
      onPress={handleLogin}
      disabled={loading}
    >
      {loading ? (
        <ActivityIndicator color="#FFFFFF" />
      ) : (
        <Text style={styles.loginButtonText}>Entrar</Text>
      )}
    </TouchableOpacity>

    <TouchableOpacity
      style={styles.signupButton}
      onPress={() => navigation.navigate('Register')}
    >
      <Text style={styles.signupText}>
        Não tem uma conta?{' '}
        <Text style={styles.signupLink}>Cadastre-se</Text>
      </Text>
    </TouchableOpacity>
  </View>
</ScrollView>
</KeyboardAvoidingView>
</SafeAreaView>
);
};

export default LoginScreen;

```


 ./src/screens/addService/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F9F9F9',
  },
  scrollContainer: {
    padding: 20,
  },
  label: {
    fontSize: 16,
    color: '#333',
    marginBottom: 8,
    fontWeight: '500',
  },
  input: {
    backgroundColor: '#FFFFFF',
    borderColor: '#CCCCCC',
    borderWidth: 1,
    borderRadius: 8,
    paddingHorizontal: 15,
    fontSize: 16,
    height: 50,
    marginBottom: 20,
  },
  textArea: {
    height: 120,
    textAlignVertical: 'top',
    paddingTop: 15,
  },
  pickerContainer: {
    backgroundColor: '#FFFFFF',
    borderColor: '#CCCCCC',
    borderWidth: 1,
    borderRadius: 8,
    marginBottom: 20,
  },
  imageUploadContainer: {
    marginBottom: 20,
  },
  imagePreviewContainer: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    justifyContent: 'flex-start',
  },
  imageWrapper: {
    position: 'relative',
    marginRight: 10,
    marginBottom: 10,
  },
});
```

```
},
imagePreview: {
  width: 80,
  height: 80,
  borderRadius: 8,
  borderWidth: 1,
  borderColor: '#E0E0E0',
  backgroundColor: '#F0F0F0',
},
imageRemoveButton: {
  position: 'absolute',
  top: -8,
  right: -8,
  backgroundColor: 'rgba(0,0,0,0.7)',
  borderRadius: 15,
  width: 25,
  height: 25,
  justifyContent: 'center',
  alignItems: 'center',
},
imageRemoveButtonText: {
  color: '#FFFFFF',
  fontWeight: 'bold',
  fontSize: 14,
  lineHeight: 16,
},
addPhotoButton: {
  width: 80,
  height: 80,
  borderRadius: 8,
  backgroundColor: '#EFEFEF',
  justifyContent: 'center',
  alignItems: 'center',
  borderWidth: 1,
  borderColor: '#CCC',
  borderStyle: 'dashed',
},
button: {
  backgroundColor: '#3F83F8',
  paddingVertical: 15,
  borderRadius: 8,
  alignItems: 'center',
  marginTop: 20,
  marginBottom: 30,
},
buttonText: {
  color: '#FFFFFF',
  fontSize: 18,
  fontWeight: 'bold',
},
});
```

 ./src/screens/addService/index.tsx

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  SafeAreaView,
  TextInput,
  TouchableOpacity,
  Alert,
  ScrollView,
  Image,
  ActivityIndicator,
} from 'react-native';
import { supabase } from '../../services/supabase';
import { styles } from './style';
import { Picker } from '@react-native-picker/picker';
import * as ImagePicker from 'expo-image-picker';
import { Ionicons } from '@expo/vector-icons';
import { decode } from 'base64-arraybuffer';

type Category = {
  id: number;
  name: string;
};

type SelectedImage = {
  uri: string;
  base64: string;
};

/**
 * Tela para adicionar um novo serviço.
 * Permite ao usuário preencher informações do serviço, selecionar uma
 * categoria,
 * e fazer upload de até 4 imagens.
 */
const AddServiceScreen = () => {
  // --- Estados ---
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [price, setPrice] = useState('');
  const [availability, setAvailability] = useState('');
  const [selectedCategory, setSelectedCategory] = useState<number | null>(
    null);
  const [categories, setCategories] = useState<Category[]>([]);
  const [loading, setLoading] = useState(false);
  const [selectedImages, setSelectedImages] = useState<SelectedImage[]>(
    []);

  useEffect(() => {
    /** Busca as categorias de serviço da base de dados ao montar o
```

```
componente. */
const fetchCategories = async () => {
  const { data, error } = await supabase
    .from('categories')
    .select('id, name')
    .order('name', { ascending: true });

  if (error) {
    Alert.alert('Erro', 'Não foi possível carregar as categorias.');
```

categoria

```
  } else if (data) {
    setCategories(data);
    if (data.length > 0 && selectedCategory === null) {
      setSelectedCategory(data[0].id); // Pré-seleciona a primeira
    }
  }
};
fetchCategories();
}, []);

/**
 * Abre a galeria de imagens para o usuário selecionar fotos.
 * Limita a seleção a 4 imagens no total.
 */
const pickImage = async () => {
  if (selectedImages.length >= 4) {
    Alert.alert('Limite atingido', 'Você só pode adicionar até 4
fotos.');
```

fotos.

```
    return;
  }

  const { status } = await
ImagePicker.requestMediaLibraryPermissionsAsync();
  if (status !== 'granted') {
    Alert.alert(
      'Permissão necessária',
      'Precisamos de acesso à sua galeria para adicionar fotos.',
    );
    return;
  }

  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.7,
    base64: true,
  });

  if (!result.canceled && result.assets && result.assets[0].base64) {
    const asset = result.assets[0];
    setSelectedImages(prevImages => [
      ...prevImages,
      { uri: asset.uri, base64: asset.base64! },
    ]
  )
}
```

```
    ]);
  }
};

/**
 * Remove uma imagem da lista de imagens selecionadas.
 * @param uriToRemove A URI da imagem a ser removida.
 */
const handleRemoveImage = (uriToRemove: string) => {
  setSelectedImages(prevImages =>
    prevImages.filter(image => image.uri !== uriToRemove),
  );
};

/**
 * Valida os dados, faz upload das imagens e salva o novo serviço no
 * Supabase.
 */
const handleSaveService = async () => {
  if (!title.trim() || !selectedCategory) {
    Alert.alert(
      'Campos obrigatórios',
      'O Título e a Categoria precisam ser preenchidos.',
    );
    return;
  }
  setLoading(true);

  try {
    const {
      data: { user },
    } = await supabase.auth.getUser();
    if (!user)
      throw new Error(
        'Usuário não autenticado. Por favor, faça login novamente.',
      );

    const { data: profileData, error: profileError } = await supabase
      .from('profiles')
      .select('location')
      .eq('id', user.id)
      .single();

    if (profileError) throw profileError;
    if (!profileData?.location) {
      Alert.alert(
        'Localização não definida',
        'Por favor, defina sua localização no seu perfil antes de cadastrar um serviço.',
      );
      setLoading(false);
      return;
    }
  }
}
```

```
const uploadedPhotoUrls: string[] = [];

for (const image of selectedImages) {
  const fileExt = image.uri.split('.').pop() || 'png';
  const fileName = `${Date.now()}.${fileExt}`;
  const filePath = `${user.id}/${fileName}`;

  const { error: uploadError } = await supabase.storage
    .from('service_images')
    .upload(filePath, decode(image.base64), {
      contentType: `image/${fileExt}`,
    });

  if (uploadError) throw uploadError;

  const { data: publicUrlData } = supabase.storage
    .from('service_images')
    .getPublicUrl(filePath);
  uploadedPhotoUrls.push(publicUrlData.publicUrl);
}

const serviceData = {
  user_id: user.id,
  title,
  description,
  category_id: selectedCategory,
  price: price ? parseFloat(price) : null,
  availability,
  photo_urls: uploadedPhotoUrls,
  location: profileData.location,
};

const { error: insertError } = await supabase
  .from('services')
  .insert(serviceData);
if (insertError) throw insertError;

Alert.alert('Sucesso!', 'Seu serviço foi cadastrado com sucesso!');

setTitle('');
setDescription('');
setPrice('');
setAvailability('');
setSelectedCategory(categories.length > 0 ? categories[0].id : null);
setSelectedImages([]);
} catch (error) {
  const errorMessage =
    error instanceof Error
      ? error.message
      : 'Ocorreu um erro desconhecido.';
  Alert.alert('Erro ao cadastrar serviço', errorMessage);
} finally {
  setLoading(false);
}
```

```

};

return (
  <SafeAreaView style={styles.container}>
    <ScrollView
      contentContainerStyle={styles.scrollContainer}
      keyboardShouldPersistTaps="handled"
    >
      <Text style={styles.label}>Título do Serviço*</Text>
      <TextInput
        style={styles.input}
        placeholder="Ex: Conserto de vazamentos"
        value={title}
        onChangeText={setTitle}
      />

      <Text style={styles.label}>Descrição</Text>
      <TextInput
        style={[styles.input, styles.textArea]}
        placeholder="Descreva o que você oferece, seus diferenciais,
etc."
        value={description}
        onChangeText={setDescription}
        multiline
      />

      <Text style={styles.label}>Categoria*</Text>
      <View style={styles.pickerContainer}>
        <Picker
          selectedValue={selectedCategory}
          onValueChange={itemValue =>
            setSelectedCategory(itemValue as number)
          }
        >
          {categories.map(category => (
            <Picker.Item
              key={category.id}
              label={category.name}
              value={category.id}
            />
          ))}
        </Picker>
      </View>

      <Text style={styles.label}>Preço (R$)</Text>
      <TextInput
        style={styles.input}
        placeholder="Ex: 50.00 (ou deixe em branco)"
        value={price}
        onChangeText={setPrice}
        keyboardType="numeric"
      />

      <Text style={styles.label}>Disponibilidade</Text>

```

```

    <TextInput
      style={styles.input}
      placeholder="Ex: Seg a Sex, 9h às 18h"
      value={availability}
      onChangeText={setAvailability}
    />

    <View style={styles.imageUploadContainer}>
      <Text style={styles.label}>Fotos do Serviço (até 4)</Text>
      <View style={styles.imagePreviewContainer}>
        {selectedImages.map(image => (
          <View key={image.uri} style={styles.imageWrapper}>
            <Image
              source={{ uri: image.uri }}
              style={styles.imagePreview}
            />
            <TouchableOpacity
              onPress={() => handleRemoveImage(image.uri)}
              style={styles.imageRemoveButton}
            >
              <Text style={styles.imageRemoveButtonText}>x</Text>
            </TouchableOpacity>
          </View>
        ))}
        {selectedImages.length < 4 && (
          <TouchableOpacity
            onPress={pickImage}
            style={styles.addPhotoButton}
          >
            <Ionicons name="add-circle-outline" size={40} color="#666"
          />
          </TouchableOpacity>
        )}
      </View>
    </View>

    <TouchableOpacity
      style={styles.button}
      onPress={handleSaveService}
      disabled={loading}
    >
      {loading ? (
        <ActivityIndicator color="#FFFFFF" />
      ) : (
        <Text style={styles.buttonText}>Salvar Serviço</Text>
      )}
    </TouchableOpacity>
  </ScrollView>
</SafeAreaView>
);
};

export default AddServiceScreen;

```


📁 ./src/screens/conversations/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'FFFFFF',
  },
  conversationItem: {
    flexDirection: 'row',
    alignItems: 'center',
    padding: 15,
    borderBottomWidth: 1,
    borderBottomColor: 'EEE',
  },
  avatar: {
    width: 50,
    height: 50,
    borderRadius: 25,
    marginRight: 15,
    backgroundColor: 'EFEFEF',
  },
  textContainer: {
    flex: 1,
  },
  userName: {
    fontSize: 16,
    fontWeight: 'bold',
  },
  lastMessage: {
    fontSize: 14,
    color: '666',
    marginTop: 2,
  },
  timestamp: {
    fontSize: 12,
    color: '999',
    marginLeft: 10,
  },
  emptyContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  emptyText: {
    fontSize: 16,
    color: '666',
    textAlign: 'center',
  },
});
```

```
    },  
  });  
};
```

📁 ./src/screens/conversations/index.tsx

```
import React, { useState, useCallback } from 'react';  
import {  
  View,  
  Text,  
  SafeAreaView,  
  FlatList,  
  TouchableOpacity,  
  Alert,  
  ActivityIndicator,  
  Image,  
} from 'react-native';  
import { useFocusEffect, useNavigation } from '@react-navigation/native';  
import { NativeStackNavigationProp } from '@react-navigation/native-stack';  
import { supabase } from '../../../services/supabase';  
import type {  
  AppStackParamList,  
  Profile as RecipientProfile,  
} from '../../../navigation/types';  
import { styles } from './style';  
import { formatDistanceToNow } from 'date-fns';  
import { ptBR } from 'date-fns/locale';  
  
/**  
 * Representa a estrutura de dados de uma única conversa retornada pela  
 API.  
 */  
type Conversation = {  
  id: number;  
  other_participant_id: string;  
  other_participant_full_name: string;  
  other_participant_avatar_url: string;  
  last_message_content: string | null;  
  last_message_at: string | null;  
};  
type ConversationsNavigationProp =  
NativeStackNavigationProp<AppStackParamList>;  
  
/**  
 * Hook para gerenciar a busca e as atualizações em tempo real das  
 conversas do usuário.  
 */  
const useConversations = () => {  
  const [conversations, setConversations] = useState<Conversation[]>([]);  
  const [loading, setLoading] = useState(true);
```

```

const fetchConversations = useCallback(async () => {
  try {
    const { data, error } = await supabase.rpc('get_user_conversations');
    if (error) throw error;
    setConversations(data || []);
  } catch (error) {
    const message =
      error instanceof Error
        ? error.message
        : 'Ocorreu um erro desconhecido.';
    Alert.alert('Erro ao carregar conversas', message);
  }
}, []);

useFocusEffect(
  useCallback(() => {
    setLoading(true);
    fetchConversations().finally(() => setLoading(false));
  }, [fetchConversations]),
);

React.useEffect(() => {
  const channel = supabase
    .channel('public:messages')
    .on(
      'postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'messages' },
      () => {
        fetchConversations();
      },
    )
    .subscribe();

  return () => {
    supabase.removeChannel(channel);
  };
}, [fetchConversations]);

return { conversations, loading };
};

/**
 * Tela que exibe a lista de conversas ativas do usuário.
 */
const ConversationsScreen = () => {
  const navigation = useNavigation<ConversationsNavigationProp>();
  const { conversations, loading } = useConversations();

  const renderItem = ({ item }: { item: Conversation }) => {
    const recipient: RecipientProfile = {
      id: item.other_participant_id,
      full_name: item.other_participant_full_name,
      avatar_url: item.other_participant_avatar_url,
    };

```

```

return (
  <TouchableOpacity
    style={styles.conversationItem}
    onPress={() =>
      navigation.navigate('Chat', {
        conversationId: item.id,
        recipient,
      })
    }
  >
    <Image
      source={{ uri: item.other_participant_avatar_url }}
      style={styles.avatar}
    />
    <View style={styles.textContainer}>
      <Text style={styles.userName}>
        {item.other_participant_full_name}
      </Text>
      <Text style={styles.lastMessage} numberOfLines={1}>
        {item.last_message_content || 'Inicie a conversa!'}
      </Text>
    </View>
    {item.last_message_at && (
      <Text style={styles.timestamp}>
        {formatDistanceToNow(new Date(item.last_message_at), {
          locale: ptBR,
          addSuffix: true,
        })}
      </Text>
    )}
  </TouchableOpacity>
);

if (loading) {
  return <ActivityIndicator style={{ flex: 1, justifyContent: 'center' }} />;
}

return (
  <SafeAreaView style={styles.container}>
    <FlatList
      data={conversations}
      keyExtractor={item => item.id.toString()}
      renderItem={renderItem}
      contentContainerStyle={{ flexGrow: 1 }}
      ListEmptyComponent={
        <View style={styles.emptyContainer}>
          <Text style={styles.emptyText}>
            Você ainda não tem conversas. Inicie uma a partir da página
            de um
            serviço!
          </Text>
        </View>
      }
    >

```

```
        </View>
      }
    />
  </SafeAreaView>
);
};

export default ConversationsScreen;
```

📁 ./src/screens/welcome/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#FFFFFF',
    padding: 20,
  },
  contentContainer: {
    flexGrow: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  logo: {
    width: 120,
    height: 120,
    resizeMode: 'contain',
    marginBottom: 24,
  },
  title: {
    fontSize: 32,
    fontWeight: 'bold',
    color: '#2D3748',
    textAlign: 'center',
    marginBottom: 12,
  },
  subtitle: {
    fontSize: 18,
    color: '#718096',
    textAlign: 'center',
    lineHeight: 26,
    paddingHorizontal: 20,
  },
  footer: {
    width: '100%',
    paddingVertical: 10,
  },
  button: {
    paddingVertical: 16,
```

```

    borderRadius: 12,
    width: '100%',
    alignItems: 'center',
    marginBottom: 16,
    elevation: 2,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.2,
    shadowRadius: 2,
  },
  buttonPrimary: {
    backgroundColor: '#3F83F8',
  },
  buttonSecondary: {
    backgroundColor: 'FFFFFF',
    borderWidth: 1.5,
    borderColor: '#E2E8F0',
  },
  buttonTextPrimary: {
    color: 'FFFFFF',
    fontSize: 18,
    fontWeight: '600',
  },
  buttonTextSecondary: {
    color: '#2D3748',
    fontSize: 18,
    fontWeight: '600',
  },
},
});

```

📁 ./src/screens/welcome/index.tsx

```

import React from 'react';
import { View, Text, Image, TouchableOpacity, StatusBar } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { styles } from './style';

// CORREÇÃO: Importamos o tipo do nosso arquivo central 'types.ts'
import { AuthStackParamList } from '../../navigation/types';

// O tipo da prop de navegação agora usa a definição importada e correta
type WelcomeScreenNavigationProp = NativeStackNavigationProp<
  AuthStackParamList,
  'Welcome'
>;

/**

```

```

* @description
* A tela de boas-vindas é a primeira tela que um usuário não autenticado
vê.
* Ela apresenta o propósito do aplicativo e fornece as ações primárias:
* fazer login ou criar uma nova conta.
* @returns {React.FC} O componente da tela de boas-vindas.
*/
const WelcomeScreen: React.FC = () => {
  const navigation = useNavigation<WelcomeScreenNavigationProp>();

  /**
   * Navega para a tela de Login.
   * Esta função encapsula a ação de navegação para melhorar a legibilidade
   * e facilitar futuras manutenções (ex: adicionar um evento de
analytics).
   */
  const navigateToLogin = () => {
    navigation.navigate('Login');
  };

  /**
   * Navega para a tela de Cadastro (Register).
   */
  const navigateToRegister = () => {
    navigation.navigate('Register');
  };

  return (
    <SafeAreaView style={styles.container}>
      <StatusBar barStyle="dark-content" backgroundColor="#FFFFFF" />
      <View style={styles.contentContainer}>
        <Image source={require('../assets/logo.png')} style=
{styles.logo} />
        <Text style={styles.title}>Sua comunidade, conectada.</Text>
        <Text style={styles.subtitle}>
          Encontre ajuda ou ofereça seus talentos. Fortaleça a economia
local
          com um toque.
        </Text>
      </View>

      <View style={styles.footer}>
        <TouchableOpacity
          style={[styles.button, styles.buttonPrimary]}
          onPress={navigateToLogin}
          activeOpacity={0.8}
        >
          <Text style={styles.buttonTextPrimary}>Fazer Login</Text>
        </TouchableOpacity>

        <TouchableOpacity
          style={[styles.button, styles.buttonSecondary]}
          onPress={navigateToRegister}
          activeOpacity={0.8}

```

```
        <Text style={styles.buttonTextSecondary}>Criar Conta</Text>
      </TouchableOpacity>
    </View>
  </SafeAreaView>
);
};

export default WelcomeScreen;
```

📁 ./src/screens/register/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: 'FFFFFF',
  },
  scrollContainer: {
    flexGrow: 1,
    justifyContent: 'center',
    padding: 20,
  },
  headerContainer: {
    alignItems: 'center',
    marginBottom: 40,
  },
  title: {
    fontSize: 32,
    fontWeight: 'bold',
    color: '#2D3748',
    marginBottom: 8,
  },
  subtitle: {
    fontSize: 18,
    color: '#718096',
    textAlign: 'center',
  },
  formContainer: {
    width: '100%',
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#F7FAFC',
    borderWidth: 1,
    borderColor: '#E2E8F0',
    borderRadius: 8,
    marginBottom: 15,
```



```
      paddingHorizontal: 15,
    },
    input: {
      flex: 1,
      height: 50,
      fontSize: 16,
      color: '#2D3748',
    },
    icon: {
      marginLeft: 10,
    },
    footerContainer: {
      width: '100%',
      alignItems: 'center',
      marginTop: 20,
    },
    registerButton: {
      width: '100%',
      backgroundColor: '#3F83F8',
      paddingVertical: 16,
      borderRadius: 12,
      alignItems: 'center',
    },
    registerButtonText: {
      color: 'FFFFFF',
      fontSize: 18,
      fontWeight: '600',
    },
    loginButton: {
      marginTop: 20,
    },
    loginText: {
      fontSize: 16,
      color: '#718096',
    },
    loginLink: {
      color: '#3F83F8',
      fontWeight: 'bold',
    },
    errorText: {
      color: '#E53E3E',
      textAlign: 'center',
      marginBottom: 10,
      fontSize: 14,
    },
  },
});
```

 ./src/screens/register/index.tsx

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  SafeAreaView,
  ActivityIndicator,
  ScrollView,
  KeyboardAvoidingView,
  Platform,
  StatusBar,
} from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { Ionicons } from '@expo/vector-icons';
import { supabase } from '../../services/supabase';
import { styles } from './style';
import { AuthStackParamList } from '../../navigation/types';

type RegisterScreenNavigationProp = NativeStackNavigationProp<
  AuthStackParamList,
  'Register'
>;

/**
 * @description
 * Tela de Cadastro de novos usuários.
 * Inclui validação de nome de usuário e um fluxo de confirmação por código
 * (OTP).
 * A tela se ajusta ao teclado para uma experiência de usuário fluida.
 */
const RegisterScreen: React.FC = () => {
  const navigation = useNavigation<RegisterScreenNavigationProp>();

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [username, setUsername] = useState('');

  const [isPasswordVisible, setIsPasswordVisible] = useState(false);
  const [isConfirmPasswordVisible, setIsConfirmPasswordVisible] =
    useState(false);

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  /**
   * Valida os campos e tenta criar um novo usuário no Supabase.
   * Em caso de sucesso, navega para a tela de confirmação de e-mail (OTP).
   */
  const handleSignUp = async () => {
    setError('');
  }
}
```

```
const usernameRegex = /^[a-zA-Z0-9]+$/;
if (!username.trim() || !email.trim() || !password) {
  setError('Por favor, preencha todos os campos obrigatórios.');
```

return;

```
}
if (!usernameRegex.test(username)) {
  setError(
    'Nome de usuário pode conter apenas letras e números, sem
    espaços.',
  );
  return;
}
if (password !== confirmPassword) {
  setError('As senhas não coincidem.');
```

return;

```
}

setLoading(true);

const { data, error } = await supabase.auth.signUp({
  email: email,
  password: password,
  options: { data: { username: username, full_name: username } },
});

if (error) {
  setError(error.message);
} else if (data.user) {
  navigation.navigate('ConfirmEmail', { email: email });
}

setLoading(false);
};

return (
  <SafeAreaView style={styles.safeArea}>
    <StatusBar barStyle="dark-content" />
    <KeyboardAvoidingView
      style={{ flex: 1 }}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    >
      <ScrollView
        contentContainerStyle={styles.scrollContainer}
        keyboardShouldPersistTaps="handled"
      >
        <View style={styles.headerContainer}>
          <Text style={styles.title}>Crie sua Conta</Text>
          <Text style={styles.subtitle}>
            Comece a se conectar com sua comunidade.
          </Text>
        </View>

        <View style={styles.formContainer}>
```

```
<View style={styles.inputContainer}>
  <TextInput
    style={styles.input}
    placeholder="Nome de usuário"
    value={username}
    onChangeText={text => {
      setUsername(text);
      setError('');
    }}
    autoCapitalize="none"
  />
</View>

<View style={styles.inputContainer}>
  <TextInput
    style={styles.input}
    placeholder="E-mail"
    value={email}
    onChangeText={text => {
      setEmail(text);
      setError('');
    }}
    keyboardType="email-address"
    autoCapitalize="none"
  />
</View>

<View style={styles.inputContainer}>
  <TextInput
    style={styles.input}
    placeholder="Senha"
    value={password}
    onChangeText={text => {
      setPassword(text);
      setError('');
    }}
    secureTextEntry={!isPasswordVisible}
  />
  <TouchableOpacity
    onPress={() => setIsPasswordVisible(!isPasswordVisible)}
  >
    <Ionicons
      name={isPasswordVisible ? 'eye-off' : 'eye'}
      size={24}
      color="#A0AEC0"
      style={styles.icon}
    />
  </TouchableOpacity>
</View>

<View style={styles.inputContainer}>
  <TextInput
    style={styles.input}
    placeholder="Confirmar Senha"
```

```
        value={confirmPassword}
        onChangeText={text => {
          setConfirmPassword(text);
          setError('');
        }}
        secureTextEntry={!isConfirmPasswordVisible}
      />
    <TouchableOpacity
      onPress={() =>
        setIsConfirmPasswordVisible(!isConfirmPasswordVisible)
      }
    >
      <Ionicons
        name={isConfirmPasswordVisible ? 'eye-off' : 'eye'}
        size={24}
        color="#A0AEC0"
        style={styles.icon}
      />
    </TouchableOpacity>
  </View>
</View>

{error ? <Text style={styles.errorText}>{error}</Text> : null}

<View style={styles.footerContainer}>
  <TouchableOpacity
    style={styles.registerButton}
    onPress={handleSignUp}
    disabled={loading}
  >
    {loading ? (
      <ActivityIndicator color="#FFFFFF" />
    ) : (
      <Text style={styles.registerButtonText}>Criar Conta</Text>
    )}
  </TouchableOpacity>

  <TouchableOpacity
    style={styles.loginButton}
    onPress={() => navigation.navigate('Login')}
  >
    <Text style={styles.loginText}>
      Já tem uma conta?{' '}
      <Text style={styles.loginLink}>Faça Login</Text>
    </Text>
  </TouchableOpacity>
</View>
</ScrollView>
</KeyboardAvoidingView>
</SafeAreaView>
);
};
```

```
export default RegisterScreen;
```

📁 ./src/screens/completeProfile/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#FFFFFF',
  },
  scrollContainer: {
    flexGrow: 1,
    justifyContent: 'center',
    padding: 20,
  },
  headerContainer: {
    alignItems: 'center',
    marginBottom: 30,
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    color: '#2D3748',
    marginBottom: 8,
  },
  subtitle: {
    fontSize: 16,
    color: '#718096',
    textAlign: 'center',
  },
  avatarPicker: {
    alignItems: 'center',
    marginBottom: 30,
  },
  avatarPlaceholder: {
    width: 120,
    height: 120,
    borderRadius: 60,
    backgroundColor: '#F7FAFC',
    borderWidth: 2,
    borderColor: '#E2E8F0',
    justifyContent: 'center',
    alignItems: 'center',
  },
  avatarImage: {
    width: 120,
    height: 120,
    borderRadius: 60,
  },
});
```

```
avatarEditText: {
  marginTop: 10,
  color: '#3F83F8',
  fontSize: 16,
  fontWeight: '500',
},
formContainer: {
  width: '100%',
},
label: {
  fontSize: 16,
  color: '#4A5568',
  marginBottom: 8,
  fontWeight: '500',
},
input: {
  height: 50,
  backgroundColor: '#F7FAFC',
  borderWidth: 1,
  borderColor: '#E2E8F0',
  borderRadius: 8,
  paddingHorizontal: 15,
  fontSize: 16,
  color: '#2D3748',
  marginBottom: 15,
},
locationButton: {
  flexDirection: 'row',
  alignItems: 'center',
  height: 50,
  backgroundColor: '#F7FAFC',
  borderWidth: 1,
  borderColor: '#E2E8F0',
  borderRadius: 8,
  paddingHorizontal: 15,
  marginBottom: 15,
},
locationText: {
  flex: 1,
  fontSize: 16,
  color: '#2D3748',
},
locationPlaceholder: {
  flex: 1,
  fontSize: 16,
  color: '#A0AEC0',
},
button: {
  width: '100%',
  backgroundColor: '#3F83F8',
  paddingVertical: 16,
  borderRadius: 12,
  alignItems: 'center',
  marginTop: 20,
```

```

    },
    buttonText: {
      color: '#FFFFFF',
      fontSize: 18,
      fontWeight: '600',
    },
  });

```

📁 ./src/screens/completeProfile/index.tsx

```

import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  SafeAreaView,
  ActivityIndicator,
  ScrollView,
  KeyboardAvoidingView,
  Platform,
  Alert,
  Image,
} from 'react-native';
import MaskInput from 'react-native-mask-input';
import * as ImagePicker from 'expo-image-picker';
import { Ionicons } from '@expo/vector-icons';
import { decode } from 'base64-arraybuffer';
import { supabase } from '../../../services/supabase';
import LocationSelectorModal from '../../../components/LocationSelectorModal';
import { styles } from './style';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
// CORREÇÃO: Importação do tipo do arquivo central 'types'
import type { AppStackParamList } from '../../../navigation/types';

type NavigationProp = NativeStackNavigationProp<
  AppStackParamList,
  'CompleteProfile'
>;

/**
 * Tela de "Completar Cadastro" (Onboarding).
 * Coleta informações essenciais do perfil do usuário, como nome, telefone,
 * localização e foto, após a confirmação do e-mail inicial.
 */
const CompleteProfileScreen: React.FC = () => {
  const navigation = useNavigation<NavigationProp>();

  const [fullName, setFullName] = useState('');

```



```
const [phone, setPhone] = useState('');
const [location, setLocation] = useState('');
const [avatarUrl, setAvatarUrl] = useState<string | null>(null);
const [loading, setLoading] = useState(false);
const [isLocationModalVisible, setLocationModalVisible] =
useState(false);

/**
 * Lida com a seleção de uma imagem de avatar, seja da câmera ou da
galeria.
 * @param source Define a origem da imagem ('camera' ou 'gallery').
 */
const handleAvatarSelection = async (source: 'camera' | 'gallery') => {
  const options: ImagePicker.ImagePickerOptions = {
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [1, 1],
    quality: 0.7,
    base64: true,
  };

  let result;
  if (source === 'camera') {
    const { status } = await ImagePicker.requestCameraPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert(
        'Permissão negada',
        'Precisamos de acesso à câmera para tirar uma foto.',
      );
      return;
    }
    result = await ImagePicker.launchCameraAsync(options);
  } else {
    const { status } =
      await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert(
        'Permissão negada',
        'Precisamos de acesso à galeria para escolher uma foto.',
      );
      return;
    }
    result = await ImagePicker.launchImageLibraryAsync(options);
  }

  if (!result.canceled && result.assets && result.assets[0].base64) {
    uploadAvatar(result.assets[0].base64);
  }
};

/**
 * Faz o upload da imagem do avatar para o Supabase Storage e atualiza o
perfil.
 * @param base64 A imagem codificada em base64.
 */
```

```
*/
const uploadAvatar = async (base64: string) => {
  setLoading(true);
  try {
    const {
      data: { user },
    } = await supabase.auth.getUser();
    if (!user) throw new Error('Usuário não encontrado');

    const filePath = `${user.id}/${new Date().getTime()}.png`;
    const imageBuffer = decode(base64);

    const { error: uploadError } = await supabase.storage
      .from('avatars')
      .upload(filePath, imageBuffer, { contentType: 'image/png' });

    if (uploadError) throw uploadError;

    const { data: publicUrlData } = supabase.storage
      .from('avatars')
      .getPublicUrl(filePath);
    const publicUrl = `${publicUrlData.publicUrl}?t=${new
Date().getTime()}`; // Cache busting
    setAvatarUrl(publicUrl);

    const { error: updateError } = await supabase
      .from('profiles')
      .update({ avatar_url: publicUrl })
      .eq('id', user.id);

    if (updateError) throw updateError;
  } catch (error) {
    const errorMessage =
      error instanceof Error
        ? error.message
        : 'Ocorreu um erro desconhecido.';
    Alert.alert('Erro no upload', errorMessage);
  } finally {
    setLoading(false);
  }
};

/**
 * Valida e salva as informações do perfil no banco de dados.
 * Em caso de sucesso, reseta a navegação para a tela principal do app.
 */
const handleCompleteProfile = async () => {
  if (!fullName.trim() || !phone || !location) {
    Alert.alert(
      'Campos obrigatórios',
      'Por favor, preencha todas as informações para continuar.',
    );
    return;
  }
}
```

```
setLoading(true);
try {
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error('Usuário não autenticado.');
```



```
  const updates = {
    id: user.id,
    full_name: fullName.trim(),
    phone,
    location,
    updated_at: new Date(),
  };

  const { error } = await supabase
    .from('profiles')
    .update(updates)
    .eq('id', user.id);
  if (error) throw error;

  navigation.reset({
    index: 0,
    routes: [{ name: 'MainTabs' }],
  });
} catch (error) {
  const errorMessage =
    error instanceof Error
      ? error.message
      : 'Ocorreu um erro desconhecido.';
  Alert.alert('Erro ao salvar perfil', errorMessage);
} finally {
  setLoading(false);
}
};

const showImagePickerOptions = () => {
  Alert.alert('Adicionar Foto', 'Escolha uma opção', [
    { text: 'Tirar Foto', onPress: () => handleAvatarSelection('camera')
  },
    {
      text: 'Escolher da Galeria',
      onPress: () => handleAvatarSelection('gallery'),
    },
    { text: 'Cancelar', style: 'cancel' },
  ]);
};

return (
  <SafeAreaView style={styles.safeArea}>
    <KeyboardAvoidingView
      style={{ flex: 1 }}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
```

```

    >
    <ScrollView
      contentContainerStyle={styles.scrollContainer}
      keyboardShouldPersistTaps="handled"
    >
      <LocationSelectorModal
        visible={isLocationModalVisible}
        onClose={() => setLocationModalVisible(false)}
        onLocationSelect={setLocation}
      />

      <View style={styles.headerContainer}>
        <Text style={styles.title}>Continue seu cadastro</Text>
        <Text style={styles.subtitle}>
          Falta pouco! Complete seu perfil para começar a usar a
plataforma.
        </Text>
      </View>

      <TouchableOpacity
        style={styles.avatarPicker}
        onPress={showImagePickerOptions}
      >
        {avatarUrl ? (
          <Image source={{ uri: avatarUrl }} style={styles.avatarImage}
        />

          ) : (
            <View style={styles.avatarPlaceholder}>
              <Ionicons name="camera" size={40} color="#A0AEC0" />
            </View>
          )}
        <Text style={styles.avatarEditText}>Adicionar foto de
perfil</Text>
      </TouchableOpacity>

      <View style={styles.formContainer}>
        <Text style={styles.label}>Nome Completo*</Text>
        <TextInput
          style={styles.input}
          placeholder="Digite seu nome e sobrenome"
          value={fullName}
          onChangeText={setFullName}
        />

        <Text style={styles.label}>Telefone (WhatsApp)*</Text>
        <MaskInput
          style={styles.input}
          value={phone}
          onChangeText={(masked, unmasked) => setPhone(unmasked)}
          mask={['( ',
            /\d/,
            /\d/,
            ') ',

```

```

        ' ',
        /\d/,
        /\d/,
        /\d/,
        /\d/,
        /\d/,
        ' ',
        /\d/,
        /\d/,
        /\d/,
        /\d/,
    ]}
    placeholder="(XX) XXXXX-XXXX"
    keyboardType="numeric"
  />

  <Text style={styles.label}>Sua Localização*</Text>
  <TouchableOpacity
    style={styles.locationButton}
    onPress={() => setLocationModalVisible(true)}
  >
    <Text
      style={
        location ? styles.locationText :
styles.locationPlaceholder
      }
    >
      {location || 'Selecione sua cidade'}
    </Text>
    <Icons name="chevron-down" size={20} color="#666" />
  </TouchableOpacity>
</View>

<TouchableOpacity
  style={styles.button}
  onPress={handleCompleteProfile}
  disabled={loading}
>
  {loading ? (
    <ActivityIndicator color="#FFFFFF" />
  ) : (
    <Text style={styles.buttonText}>Finalizar Cadastro</Text>
  )}
</TouchableOpacity>
</ScrollView>
</KeyboardAvoidingView>
</SafeAreaView>
);
};

export default CompleteProfileScreen;

```

 ./src/screens/resetPassword/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: { flex: 1, backgroundColor: '#FFFFFF' },
  container: { flex: 1, justifyContent: 'center', padding: 20 },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#2D3748',
    textAlign: 'center',
    marginBottom: 15,
  },
  subtitle: {
    fontSize: 16,
    color: '#718096',
    textAlign: 'center',
    marginBottom: 30,
    lineHeight: 24,
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#F7FAFC',
    borderWidth: 1,
    borderColor: '#E2E8F0',
    borderRadius: 8,
    marginBottom: 15,
    paddingHorizontal: 15,
  },
  input: { flex: 1, height: 50, fontSize: 16, color: '#2D3748' },
  button: {
    width: '100%',
    backgroundColor: '#3F83F8',
    paddingVertical: 16,
    borderRadius: 12,
    alignItems: 'center',
  },
  buttonText: { color: '#FFFFFF', fontSize: 18, fontWeight: '600' },
  errorText: {
    color: '#E53E3E',
    textAlign: 'center',
    marginBottom: 10,
    fontSize: 14,
  },
});
```

 ./src/screens/resetPassword/index.tsx

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  SafeAreaView,
  ActivityIndicator,
  Alert,
  StatusBar,
} from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { Ionicons } from '@expo/vector-icons';
import { supabase } from '../../services/supabase';
import { styles } from '../../resetPassword/style';
import { AuthStackParamList } from '../../navigation/types';

type NavigationProp = NativeStackNavigationProp<
  AuthStackParamList,
  'ResetPassword'
>;

/**
 * @description
 * Tela onde o usuário define a nova senha após clicar no link de
recuperação de e-mail.
 * O usuário é autenticado temporariamente pelo token da URL.
 */
const ResetPasswordScreen: React.FC = () => {
  const navigation = useNavigation<NavigationProp>();
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [isPasswordVisible, setIsPasswordVisible] = useState(false);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  /**
   * Valida e atualiza a senha do usuário no Supabase.
   * Em caso de sucesso, reseta a navegação para a tela de Login.
   */
  const handleResetPassword = async () => {
    setError('');
    if (!password) {
      setError('Por favor, digite uma nova senha.');
```

```
});

if (error) {
  setError(error.message);
} else {
  Alert.alert(
    'Senha Redefinida!',
    'Sua senha foi alterada com sucesso. Por favor, faça o login novamente.',
  );
  navigation.reset({ index: 0, routes: [{ name: 'Login' }] });
}
setLoading(false);
};

return (
  <SafeAreaView style={styles.safeArea}>
    <StatusBar barStyle="dark-content" />
    <View style={styles.container}>
      <Text style={styles.title}>Defina sua Nova Senha</Text>
      <Text style={styles.subtitle}>Escolha uma senha forte e segura.</Text>

      <View style={styles.inputContainer}>
        <TextInput
          style={styles.input}
          placeholder="Nova senha"
          value={password}
          onChangeText={text => {
            setPassword(text);
            setError('');
          }}
          secureTextEntry={!isPasswordVisible}
        />
        <TouchableOpacity
          onPress={() => setIsPasswordVisible(!isPasswordVisible)}
        >
          <Ionicons
            name={isPasswordVisible ? 'eye-off' : 'eye'}
            size={24}
            color="#A0AEC0"
          />
        </TouchableOpacity>
      </View>

      <View style={styles.inputContainer}>
        <TextInput
          style={styles.input}
          placeholder="Confirmar nova senha"
          value={confirmPassword}
          onChangeText={text => {
            setConfirmPassword(text);
            setError('');
          }}
        >>
      </View>
    </View>
  </SafeAreaView>
);
```



```

        secureTextEntry={!isPasswordVisible}
      />
    </View>

    {error ? <Text style={styles.errorText}>{error}</Text> : null}

    <TouchableOpacity
      style={styles.button}
      onPress={handleResetPassword}
      disabled={loading}
    >
      {loading ? (
        <ActivityIndicator color="#FFFFFF" />
      ) : (
        <Text style={styles.buttonText}>Salvar Nova Senha</Text>
      )}
    </TouchableOpacity>
  </View>
</SafeAreaView>
);
};

export default ResetPasswordScreen;

```

📁 ./src/screens/changePassword/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#F9F9F9',
  },
  headerText: {
    fontSize: 16,
    color: '#666',
    marginBottom: 20,
    textAlign: 'center',
    lineHeight: 24,
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: 'FFFFFF',
    borderColor: 'CCCCCC',
    borderWidth: 1,
    borderRadius: 8,
    marginBottom: 15,
    paddingHorizontal: 15,
  },

```

```
,
input: {
  flex: 1,
  height: 50,
  fontSize: 16,
  color: '#333',
},
icon: {
  marginLeft: 10,
},
button: {
  backgroundColor: '#3F83F8',
  paddingVertical: 15,
  borderRadius: 8,
  alignItems: 'center',
  marginTop: 10,
},
buttonDisabled: {
  backgroundColor: '#A5C8FF',
},
buttonText: {
  color: '#FFFFFF',
  fontSize: 18,
  fontWeight: 'bold',
},
errorText: {
  color: '#E53E3E',
  textAlign: 'center',
  marginBottom: 10,
  fontSize: 14,
},
modalContainer: {
  flex: 1,
  justifyContent: 'center',
  alignItems: 'center',
  backgroundColor: 'rgba(0, 0, 0, 0.5)',
},
modalView: {
  width: 150,
  height: 150,
  backgroundColor: 'white',
  borderRadius: 20,
  justifyContent: 'center',
  alignItems: 'center',
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.25,
  shadowRadius: 4,
  elevation: 5,
},
modalText: {
  marginTop: 10,
  fontSize: 16,
  fontWeight: 'bold',
},
```

```
    },  
  });  
};
```

📁 ./src/screens/changePassword/index.tsx

```
import React, { useState } from 'react';  
import {  
  View,  
  Text,  
  TextInput,  
  TouchableOpacity,  
  SafeAreaView,  
  Modal,  
  ActivityIndicator,  
  TextInputProps,  
} from 'react-native';  
import { useNavigation } from '@react-navigation/native';  
import { Ionicons } from '@expo/vector-icons';  
import { supabase } from '../../../services/supabase';  
import { styles } from './style';  
  
interface PasswordInputProps extends TextInputProps {  
  value: string;  
  onChangeText: (text: string) => void;  
}  
  
/**  
 * Um componente de input de senha reutilizável com um ícone para alternar  
 a visibilidade.  
 */  
const PasswordInput: React.FC<PasswordInputProps> = ({  
  value,  
  onChangeText,  
  ...props  
}) => {  
  const [isVisible, setIsVisible] = useState(false);  
  
  return (  
    <View style={styles.inputContainer}>  
      <TextInput  
        style={styles.input}  
        value={value}  
        onChangeText={onChangeText}  
        secureTextEntry={!isVisible}  
        autoCapitalize="none"  
        {...props}  
      />  
      <TouchableOpacity  
        onPress={() => setIsVisible(!isVisible)}  
        style={styles.icon}
```

```
        >
        <Ionicons name={isVisible ? 'eye-off' : 'eye'} size={24}
color="gray" />
      </TouchableOpacity>
    </View>
  );
};

// --- Componente Principal ---

/**
 * Tela que permite ao usuário alterar sua senha.
 * O usuário deve fornecer a senha atual e uma nova senha (com
 * confirmação).
 */
const ChangePasswordScreen = () => {
  const navigation = useNavigation();

  const [currentPassword, setCurrentPassword] = useState('');
  const [newPassword, setNewPassword] = useState('');
  const [confirmNewPassword, setConfirmNewPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [showSuccessModal, setShowSuccessModal] = useState(false);

  /**
   * Valida as senhas inseridas pelo usuário.
   * @returns Uma string de erro se a validação falhar, caso contrário,
   * null.
   */
  const validatePasswords = (): string | null => {
    if (!currentPassword || !newPassword || !confirmNewPassword) {
      return 'Todos os campos são obrigatórios.';
    }
    if (newPassword.length < 6) {
      return 'A nova senha deve ter no mínimo 6 caracteres.';
    }
    if (newPassword !== confirmNewPassword) {
      return 'As novas senhas não coincidem.';
    }
    return null;
  };

  /**
   * Orquestra o processo de alteração de senha, incluindo validação e
   * chamadas à API.
   */
  const handleChangePassword = async () => {
    setError('');
    const validationError = validatePasswords();
    if (validationError) {
      setError(validationError);
      return;
    }
  }
}
```

```
setLoading(true);
try {
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user?.email)
    throw new Error(
      'Usuário não autenticado. Por favor, faça login novamente.',
    );

  // Verifica se a senha atual está correta tentando fazer login com
  ela
  const { error: signInError } = await
supabase.auth.signInWithPassword({
  email: user.email,
  password: currentPassword,
});

  if (signInError) {
    setError('A senha atual está incorreta.');
```

return;

```
  }

  const { error: updateError } = await supabase.auth.updateUser({
    password: newPassword,
  });

  if (updateError) throw updateError;

  setShowSuccessModal(true);
  setTimeout(() => {
    setShowSuccessModal(false);
    navigation.goBack();
  }, 2000);
} catch (err) {
  const errorMessage =
err instanceof Error ? err.message : 'Ocorreu um erro
desconhecido.';
  setError(errorMessage);
} finally {
  setLoading(false);
}
};

return (
  <SafeAreaView style={styles.container}>
    <Modal transparent={true} visible={showSuccessModal}
animationType="fade">
      <View style={styles.modalContainer}>
        <View style={styles.modalView}>
          <Ionicons name="checkmark-circle" size={80} color="#48BB78" />
          <Text style={styles.modalText}>Senha Alterada!</Text>
        </View>
```

```

    </View>
  </Modal>

  <Text style={styles.headerText}>
    Sua senha precisa ter no mínimo 6 caracteres para sua segurança.
  </Text>

  <PasswordInput
    placeholder="Senha atual"
    value={currentPassword}
    onChangeText={setCurrentPassword}
  />

  <PasswordInput
    placeholder="Nova senha"
    value={newPassword}
    onChangeText={setNewPassword}
  />

  <PasswordInput
    placeholder="Repita a nova senha"
    value={confirmNewPassword}
    onChangeText={setConfirmNewPassword}
  />

  {error ? <Text style={styles.errorText}>{error}</Text> : null}

  <TouchableOpacity
    style={[styles.button, loading && styles.buttonDisabled]}
    onPress={handleChangePassword}
    disabled={loading}
  >
    {loading ? (
      <ActivityIndicator color="#FFFFFF" />
    ) : (
      <Text style={styles.buttonText}>Confirmar alteração</Text>
    )}
  </TouchableOpacity>
</SafeAreaView>
);
};

export default ChangePasswordScreen;

```

📁 ./src/screens/forgotPassword/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  safeArea: { flex: 1, backgroundColor: '#FFFFFF' },

```

```
    container: { flex: 1, justifyContent: 'center', padding: 20 },
    title: {
      fontSize: 24,
      fontWeight: 'bold',
      color: '#2D3748',
      textAlign: 'center',
      marginBottom: 15,
    },
    subtitle: {
      fontSize: 16,
      color: '#718096',
      textAlign: 'center',
      marginBottom: 30,
      lineHeight: 24,
    },
    inputContainer: {
      flexDirection: 'row',
      alignItems: 'center',
      backgroundColor: '#F7FAFC',
      borderWidth: 1,
      borderColor: '#E2E8F0',
      borderRadius: 8,
      marginBottom: 15,
      paddingHorizontal: 15,
    },
    input: { flex: 1, height: 50, fontSize: 16, color: '#2D3748' },
    button: {
      width: '100%',
      backgroundColor: '#3F83F8',
      paddingVertical: 16,
      borderRadius: 12,
      alignItems: 'center',
    },
    buttonText: { color: 'FFFFFF', fontSize: 18, fontWeight: '600' },
    errorText: {
      color: '#E53E3E',
      textAlign: 'center',
      marginBottom: 10,
      fontSize: 14,
    },
  },
});
```

 ./src/screens/forgotPassword/index.tsx

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
```

```
SafeAreaView,
ActivityIndicator,
Alert,
StatusBar,
} from 'react-native';
import { supabase } from '../../services/supabase';
import { styles } from './style';

/**
 * @description
 * Tela onde o usuário insere seu e-mail para iniciar o fluxo de
recuperação de senha.
 * Envia uma solicitação ao Supabase para que um e-mail com um link de
 * redirecionamento (deep link) seja enviado para o aplicativo.
 */
const ForgotPasswordScreen: React.FC = () => {
  const [email, setEmail] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  /**
   * Valida o e-mail e chama a função de reset de senha do Supabase.
   * A propriedade 'redirectTo' é crucial para sobrescrever a 'Site URL'
padrão
   * e garantir que o usuário seja redirecionado para o aplicativo.
   */
  const handlePasswordReset = async () => {
    setError('');
    if (!email.trim()) {
      setError('Por favor, insira seu e-mail.');
```

return;

```
    }

    setLoading(true);
    const { error } = await supabase.auth.resetPasswordForEmail(email, {
      redirectTo: 'servicoscomunitarios://reset-password',
    });

    if (error) {
      setError(error.message);
    } else {
      Alert.alert(
        'Verifique seu E-mail',
        'Se uma conta com este e-mail existir, enviamos um link para
redefinir sua senha.',
      );
    }
    setLoading(false);
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <StatusBar barStyle="dark-content" />
      <View style={styles.container}>
```



```

    <Text style={styles.title}>Recuperar Senha</Text>
    <Text style={styles.subtitle}>
      Digite seu e-mail e enviaremos um link para você voltar a acessar
sua
      conta.
    </Text>

    <View style={styles.inputContainer}>
      <TextInput
        style={styles.input}
        placeholder="Seu e-mail"
        value={email}
        onChangeText={text => {
          setEmail(text);
          setError('');
        }}
        keyboardType="email-address"
        autoCapitalize="none"
      />
    </View>

    {error ? <Text style={styles.errorText}>{error}</Text> : null}

    <TouchableOpacity
      style={styles.button}
      onPress={handlePasswordReset}
      disabled={loading}
    >
      {loading ? (
        <ActivityIndicator color="#FFFFFF" />
      ) : (
        <Text style={styles.buttonText}>Enviar Link de
Recuperação</Text>
      )}
    </TouchableOpacity>
  </View>
</SafeAreaView>
);
};

export default ForgotPasswordScreen;

```

 ./src/navigation/AppNavigator.tsx

```

import React, { useState, useEffect } from 'react';
import {
  createNativeStackNavigator,
  NativeStackScreenProps,
} from '@react-navigation/native-stack';
import { View, ActivityIndicator } from 'react-native';

```

```

import { supabase } from '../services/supabase';
import { AppStackParamList } from './types';
import TabNavigator from './TabNavigator';
import EditProfileScreen from '../screens/editProfile';
import ChangePasswordScreen from '../screens/changePassword';
import AddServiceScreen from '../screens/addService';
import ServiceDetailScreen from '../screens/serviceDetail';
import ChatScreen from '../screens/chat';
import CompleteProfileScreen from '../screens/completeProfile';

const Stack = createNativeStackNavigator<AppStackParamList>();

/**
 * @description
 * Navegador principal para usuários autenticados.
 * Este componente verifica se o perfil do usuário está completo para
 * definir a rota inicial: a tela para completar o perfil ou a tela
principal com abas.
 * @returns {React.FC} O navegador da pilha principal do app.
 */
const AppNavigator: React.FC = () => {
  const [isProfileComplete, setProfileComplete] = useState<boolean | null>(
    null,
  );

  useEffect(() => {
    const checkProfile = async () => {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      if (user) {
        const { data: profile } = await supabase
          .from('profiles')
          .select('full_name, phone, location')
          .eq('id', user.id)
          .single();

        setProfileComplete(
          !(profile && profile.full_name && profile.phone &&
profile.location),
        );
      }
    };
    checkProfile();
  }, []);

  if (isProfileComplete === null) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
        <ActivityIndicator size="large" color="#3F83F8" />
      </View>
    );
  }
}

```

```
return (  
  <Stack.Navigator  
    initialRouteName={isProfileComplete ? 'MainTabs' : 'CompleteProfile'}  
  >  
    <Stack.Screen  
      name="CompleteProfile"  
      component={CompleteProfileScreen}  
      options={{ headerShown: false }}  
    />  
    <Stack.Screen  
      name="MainTabs"  
      component={TabNavigator}  
      options={{ headerShown: false }}  
    />  
    <Stack.Screen  
      name="EditProfile"  
      component={EditProfileScreen}  
      options={{ title: 'Editar Perfil' }}  
    />  
    <Stack.Screen  
      name="ChangePassword"  
      component={ChangePasswordScreen}  
      options={{ title: 'Alterar Senha' }}  
    />  
    <Stack.Screen  
      name="AddService"  
      component={AddServiceScreen}  
      options={{ title: 'Cadastrar Serviço' }}  
    />  
    <Stack.Screen  
      name="ServiceDetail"  
      component={ServiceDetailScreen}  
      options={{ title: 'Detalhes do Serviço' }}  
    />  
    <Stack.Screen  
      name="Chat"  
      component={ChatScreen}  
      options={{(  
        route,  
      ): NativeStackScreenProps<AppStackParamList, 'Chat'> => ({  
        title: route.params.recipient.full_name,  
      })}  
    />  
  </Stack.Navigator>  
);  
};  
  
export default AppNavigator;
```

```
/**
 * @description
 * Arquivo central para a definição de tipos de navegação.
 * Segue o princípio de "Single Source of Truth" (Fonte Única da Verdade),
 * quebrando dependências circulares e organizando o código.
 */

export type Profile = {
  id: string;
  full_name: string;
  avatar_url: string | null;
};

export type AuthStackParamList = {
  Welcome: undefined;
  Login: undefined;
  Register: undefined;
  ConfirmEmail: { email: string };
  ForgotPassword: undefined;
  ResetPassword: undefined;
};

export type AppStackParamList = {
  MainTabs: undefined;
  CompleteProfile: undefined;
  EditProfile: undefined;
  ChangePassword: undefined;
  AddService: undefined;
  ServiceDetail: { serviceId: number };
  Chat: {
    conversationId: number;
    recipient: Profile;
  };
};

export type TabParamList = {
  Início: undefined;
  Pesquisar: undefined;
  Mensagens: undefined;
  Perfil: undefined;
};
```

 ./src/navigation/AuthNavigator.tsx

```
import React from 'react';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { AuthStackParamList } from '../types';
import WelcomeScreen from '../screens/welcome';
```

```
import LoginScreen from '../screens/login';
import RegisterScreen from '../screens/register';
import ConfirmEmailScreen from '../screens/confirmEmail';
import ForgotPasswordScreen from '../screens/forgotPassword';
import ResetPasswordScreen from '../screens/resetPassword';

const AuthStack = createNativeStackNavigator<AuthStackParamList>();

/**
 * @description
 * Componente de navegação que gerencia todas as telas relacionadas ao
fluxo de autenticação
 * (telas que o usuário vê antes de fazer o login).
 * @returns {React.FC} O navegador da pilha de autenticação.
 */
const AuthNavigator: React.FC = () => {
  return (
    <AuthStack.Navigator
      screenOptions={{
        headerBackTitle: '',
        headerTintColor: '#2D3748',
      }}
    >
      <AuthStack.Screen
        name="Welcome"
        component={WelcomeScreen}
        options={{ headerShown: false }}
      />
      <AuthStack.Screen
        name="Login"
        component={LoginScreen}
        options={{ title: 'Login' }}
      />
      <AuthStack.Screen
        name="Register"
        component={RegisterScreen}
        options={{ title: 'Cadastro' }}
      />
      <AuthStack.Screen
        name="ConfirmEmail"
        component={ConfirmEmailScreen}
        options={{ title: 'Confirmação Pendente' }}
      />
      <AuthStack.Screen
        name="ForgotPassword"
        component={ForgotPasswordScreen}
        options={{ title: 'Recuperar Senha' }}
      />
      <AuthStack.Screen
        name="ResetPassword"
        component={ResetPasswordScreen}
        options={{ title: 'Redefinir Senha' }}
      />
    </AuthStack.Navigator>
  );
};
```

```

    );
  };

  export default AuthNavigator;

```

📁 ./src/navigation/TabNavigator.tsx

```

import React from 'react';
import {
  createBottomTabNavigator,
  BottomTabScreenProps,
} from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';
import { TabParamList } from './types';
import HomeScreen from '../screens/home';
import SearchScreen from '../screens/search';
import ProfileScreen from '../screens/profile';
import ConversationsScreen from '../screens/conversations';

const Tab = createBottomTabNavigator<TabParamList>();

/**
 * @description
 * Componente de navegação que gerencia a barra de abas principal do
 aplicativo.
 * É a interface principal para um usuário autenticado.
 * @returns {React.FC} O navegador de abas.
 */
const TabNavigator: React.FC = () => {
  return (
    <Tab.Navigator
      screenOptions={({ route }: BottomTabScreenProps<TabParamList>) => ({
        tabBarIcon: ({ focused, color, size }) => {
          let iconName: keyof typeof Ionicons.glyphMap;

          switch (route.name) {
            case 'Início':
              iconName = focused ? 'home' : 'home-outline';
              break;
            case 'Pesquisar':
              iconName = focused ? 'search' : 'search-outline';
              break;
            case 'Mensagens':
              iconName = focused ? 'chatbubbles' : 'chatbubbles-outline';
              break;
            case 'Perfil':
              iconName = focused ? 'person' : 'person-outline';
              break;
            default:
              iconName = 'alert-circle-outline';

```

```

        break;
    }

    return <Icons name={iconName} size={size} color={color} />;
  },
  tabBarActiveTintColor: '#3F83F8',
  tabBarInactiveTintColor: 'gray',
  headerShown: false,
  }}}
>
<Tab.Screen name="Início" component={HomeScreen} />
<Tab.Screen name="Pesquisar" component={SearchScreen} />
<Tab.Screen name="Mensagens" component={ConversationsScreen} />
<Tab.Screen name="Perfil" component={ProfileScreen} />
</Tab.Navigator>
);
};

export default TabNavigator;

```

 ./src/components/PristineSearch/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2D3748',
    marginBottom: 20,
  },
  categoryContainer: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    gap: 12,
  },
  categoryChip: {
    backgroundColor: 'FFFFFF',
    paddingVertical: 10,
    paddingHorizontal: 16,
    borderRadius: 20,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  categoryChipText: {
    fontSize: 14,
  },
});

```

```
    fontWeight: '500',  
    color: '#4A5568',  
  },  
});
```

📁 ./src/components/PristineSearch/index.tsx

```
import React, { useState, useEffect } from 'react';  
import {  
  View,  
  Text,  
  TouchableOpacity,  
  ActivityIndicator,  
  Alert,  
} from 'react-native';  
import { supabase } from '../../../services/supabase';  
import { styles } from './style';  
  
type Category = {  
  id: number;  
  name: string;  
};  
  
type Props = {  
  onCategoryPress: (categoryId: number, categoryName: string) => void;  
};  
  
/**  
 * @description  
 * Componente para o estado inicial da tela de busca, exibindo categorias.  
 */  
const PristineSearch: React.FC<Props> = ({ onCategoryPress }) => {  
  const [categories, setCategories] = useState<Category[]>([]);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    const fetchCategories = async () => {  
      const { data, error } = await supabase  
        .from('categories')  
        .select('id, name')  
        .limit(8);  
  
      if (error) {  
        Alert.alert('Erro', 'Não foi possível carregar as categorias.');      } else {  
        setCategories(data || []);  
      }  
      setLoading(false);  
    };  
    fetchCategories();  
  });  
}
```



```

    }, []);

    if (loading) {
      return <ActivityIndicator style={{ marginTop: 30 }} />;
    }

    return (
      <View style={styles.container}>
        <Text style={styles.title}>Explorar Categorias</Text>
        <View style={styles.categoryContainer}>
          {categories.map(category => (
            <TouchableOpacity
              key={category.id}
              style={styles.categoryChip}
              // Aqui, nós chamamos a função com os dois argumentos
esperados.
              onPress={() => onCategoryPress(category.id, category.name)}
            >
              <Text style={styles.categoryChipText}>{category.name}</Text>
            </TouchableOpacity>
          ))}
        </View>
      </View>
    );
  };
};

export default PristineSearch;

```

📁 ./src/components/ReviewModal/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  overlay: {
    flex: 1,
    backgroundColor: 'rgba(0,0,0,0.6)',
    justifyContent: 'center',
    alignItems: 'center',
  },
  container: {
    backgroundColor: 'white',
    borderRadius: 12,
    padding: 20,
    width: '90%',
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 20,
  },

```

```

    },
    starsContainer: {
      flexDirection: 'row',
      justifyContent: 'center',
      marginBottom: 20,
    },
    commentInput: {
      backgroundColor: '#F0F0F0',
      borderRadius: 8,
      height: 100,
      padding: 10,
      textAlignVertical: 'top',
      fontSize: 16,
      marginBottom: 20,
    },
    button: {
      backgroundColor: '#3F83F8',
      padding: 15,
      borderRadius: 8,
      alignItems: 'center',
    },
    buttonText: {
      color: 'FFFFFF',
      fontSize: 16,
      fontWeight: 'bold',
    },
  },
});

```

📁 ./src/components/ReviewModal/index.tsx

```

import React, { useState } from 'react';
import {
  View,
  Text,
  Modal,
  TextInput,
  TouchableOpacity,
  ActivityIndicator,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { styles } from './style';

type Props = {
  visible: boolean;
  onClose: () => void;
  onSubmit: (rating: number, comment: string) => Promise<void>;
};

const ReviewModal = ({ visible, onClose, onSubmit }: Props) => {
  const [rating, setRating] = useState(0);

```

```

const [comment, setComment] = useState('');
const [loading, setLoading] = useState(false);

const handleSubmit = async () => {
  setLoading(true);
  await onSubmit(rating, comment);
  setLoading(false);
  setRating(0);
  setComment('');
  onClose();
};

return (
  <Modal
    animationType="fade"
    transparent={true}
    visible={visible}
    onRequestClose={onClose}
  >
    <View style={styles.overlay}>
      <View style={styles.container}>
        <Text style={styles.title}>Deixe sua Avaliação</Text>

        <View style={styles.starsContainer}>
          {[1, 2, 3, 4, 5].map(star => (
            <TouchableOpacity key={star} onPress={() => setRating(star)}>
              <Ionicons
                name={star <= rating ? 'star' : 'star-outline'}
                size={40}
                color="#FFC107"
              />
            </TouchableOpacity>
          ))}
        </View>

        <TextInput
          style={styles.commentInput}
          placeholder="Escreva seu comentário (opcional)..."
          value={comment}
          onChangeText={setComment}
          multiline
        />

        <TouchableOpacity
          style={styles.button}
          onPress={handleSubmit}
          disabled={loading}
        >
          {loading ? (
            <ActivityIndicator color="#FFF" />
          ) : (
            <Text style={styles.buttonText}>Enviar Avaliação</Text>
          )}
        </TouchableOpacity>
      </View>
    </View>
  )
);

```

```
        </View>
      </View>
    </Modal>
  );
};

export default ReviewModal;
```

📁 ./src/components/LocationSelectorModal/style.ts

```
import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  overlay: {
    flex: 1,
    backgroundColor: 'rgba(0,0,0,0.6)',
    justifyContent: 'center',
    alignItems: 'center',
  },
  container: {
    backgroundColor: 'white',
    borderRadius: 12,
    padding: 20,
    width: '90%',
    height: '80%',
  },
  inputContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#F0F0F0',
    borderRadius: 8,
    paddingHorizontal: 10,
    marginBottom: 10,
  },
  input: {
    flex: 1,
    height: 45,
    fontSize: 16,
    marginLeft: 10,
  },
  listItem: {
    paddingVertical: 15,
    borderBottomWidth: 1,
    borderBottomColor: '#EEE',
  },
  listItemText: {
    fontSize: 16,
  },
});
```

 ./src/components/LocationSelectorModal/index.tsx

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  Modal,
  TextInput,
  FlatList,
  TouchableOpacity,
  ActivityIndicator,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { styles } from './style';

type City = {
  id: number;
  nome: string;
  microrregiao: {
    mesorregiao: {
      UF: {
        sigla: string;
      };
    };
  };
};

type Props = {
  visible: boolean;
  onClose: () => void;
  onLocationSelect: (location: string) => void;
};

const LocationSelectorModal = ({
  visible,
  onClose,
  onLocationSelect,
}: Props) => {
  const [searchText, setSearchText] = useState('');
  const [cities, setCities] = useState<City[]>([]);
  const [loading, setLoading] = useState(false);

  // Efeito para buscar cidades na API do IBGE quando o texto de busca muda
  useEffect(() => {
    // Ignora a busca se o texto for muito curto
    if (searchText.length < 3) {
      setCities([]);
      return;
    }

    setLoading(true);
```

```

const fetchCities = async () => {
  try {
    const response = await fetch(
      `https://servicodados.ibge.gov.br/api/v1/localidades/municipios?
orderBy=nome`,
    );
    const allCities: City[] = await response.json();

    // Filtra as cidades no cliente para corresponder ao texto de busca
    const filtered = allCities.filter(city =>
      city.nome.toLowerCase().includes(searchText.toLowerCase()),
    );

    setCities(filtered);
  } catch (error) {
    console.error('Erro ao buscar cidades:', error);
  } finally {
    setLoading(false);
  }
};

// Debounce: Atraso para evitar muitas chamadas à API enquanto o
usuário digita
const timerId = setTimeout(() => {
  fetchCities();
}, 500); // Espera 500ms após o usuário parar de digitar

return () => clearTimeout(timerId); // Limpa o timer
}, [searchText]);

const handleSelect = (city: City) => {
  const locationString = `${city.nome},
${city.microrregiao.mesorregiao.UF.sigla}`;
  onLocationSelect(locationString);
  onClose();
};

return (
  <Modal
    animationType="fade"
    transparent={true}
    visible={visible}
    onRequestClose={onClose}
  >
    <View style={styles.overlay}>
      <View style={styles.container}>
        <View style={styles.inputContainer}>
          <Ionicons name="search" size={20} color="#999" />
          <TextInput
            style={styles.input}
            placeholder="Digite o nome da sua cidade..."
            value={searchText}
            onChangeText={setSearchText}
          />

```

```

    </View>
    {loading ? (
      <ActivityIndicator style={{ marginTop: 20 }} />
    ) : (
      <FlatList
        data={cities}
        keyExtractor={item => item.id.toString()}
        renderItem={({ item }) => (
          <TouchableOpacity
            style={styles.listItem}
            onPress={() => handleSelect(item)}
          >
            <Text style={styles.listItemText}>
              {item.nome}, {item.microrregiao.mesorregiao.UF.sigla}
            </Text>
          </TouchableOpacity>
        )}
        keyboardShouldPersistTaps="handled"
      />
    )}
  </View>
</Modal>
);
};

export default LocationSelectorModal;

```

📁 ./src/components/ServiceCard/style.ts

```

import { StyleSheet } from 'react-native';

export const styles = StyleSheet.create({
  card: {
    backgroundColor: '#FFFFFF',
    borderRadius: 12,
    marginBottom: 20,
    overflow: 'hidden',
    elevation: 3,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
  },
  image: {
    width: '100%',
    height: 150,
    backgroundColor: '#EFEFEF',
  },
  infoContainer: {

```

```

        padding: 15,
      },
      title: {
        fontSize: 18,
        fontWeight: 'bold',
        color: '#2D3748',
        marginBottom: 4,
      },
      providerName: {
        fontSize: 14,
        color: '#718096',
        marginBottom: 8,
      },
      ratingContainer: {
        flexDirection: 'row',
        alignItems: 'center',
        marginBottom: 8,
      },
      ratingText: {
        marginLeft: 6,
        fontSize: 14,
        color: '#4A5568',
      },
      price: {
        fontSize: 18,
        fontWeight: 'bold',
        color: '#3F83F8',
        marginTop: 4,
      },
    },
  });

```

 ./src/components/ServiceCard/index.tsx

```

import React from 'react';
import { View, Text, Image, TouchableOpacity } from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { Ionicons } from '@expo/vector-icons';
import { AppStackParamList } from '../../navigation/types'; // Importação corrigida
import { styles } from './style';

/**
 * @description
 * Define a estrutura de dados esperada para um card de serviço.
 * Exportar este tipo permite que outras partes do aplicativo (como a tela de busca)
 * saibam exatamente quais dados este componente precisa.
 */
export type ServiceCardData = {

```



```

    id: number;
    title: string;
    price: number | null;
    photo_urls: string[] | null;
    full_name: string | null;
    avg_rating: number | null;
    review_count: number | null;
  };

  type Props = {
    service: ServiceCardData;
  };

  type NavigationProp = NativeStackNavigationProp<AppStackParamList,
'MainTabs'>;

/**
 * @description
 * Componente reutilizável para exibir um resumo de um serviço em um card.
 * É responsável pela apresentação visual e pela navegação para a tela de
detalhes,
 * seguindo o princípio de "Single Responsibility".
 * @param {Props} props - As propriedades do componente, contendo os dados
do serviço.
 * @returns {React.FC} Um card de serviço clicável.
 */
const ServiceCard: React.FC<Props> = ({ service }) => {
  const navigation = useNavigation<NavigationProp>();

  /**
   * @description Renderiza as estrelas de avaliação com base em uma nota.
   * @param {number | null} rating - A nota média do serviço.
   * @returns {JSX.Element[]} Um array de componentes de ícone de estrela.
   */
  const renderStars = (rating: number | null) => {
    const totalStars = 5;
    const starRating = rating ? Math.round(rating * 2) / 2 : 0;
    const starElements = [];
    for (let i = 1; i <= totalStars; i++) {
      let iconName: keyof typeof Ionicons.glyphMap = 'star-outline';
      if (i <= starRating) {
        iconName = 'star';
      } else if (i - 0.5 === starRating) {
        iconName = 'star-half-sharp';
      }
      starElements.push(
        <Ionicons key={i} name={iconName} size={16} color="#FFC107" />,
      );
    }
    return starElements;
  };

  return (
    <TouchableOpacity

```

```

    style={styles.card}
    activeOpacity={0.8}
    onPress={() =>
      navigation.navigate('ServiceDetail', { serviceId: service.id })
    }
  >
  <Image
    source={{
      uri: service.photo_urls?.[0] ||
      'https://via.placeholder.com/300',
    }}
    style={styles.image}
  />
  <View style={styles.infoContainer}>
    <Text style={styles.title} numberOfLines={1}>
      {service.title}
    </Text>
    <Text style={styles.providerName}>
      por {service.full_name || 'Prestador'}
    </Text>
    <View style={styles.ratingContainer}>
      {renderStars(service.avg_rating)}
      <Text style={styles.ratingText}>
        {service.avg_rating ? service.avg_rating.toFixed(1) : 'Novo'} (
          {service.review_count || 0})
      </Text>
    </View>
    {service.price != null && (
      <Text style={styles.price}>R$ {service.price.toFixed(2)}</Text>
    )}
  </View>
</TouchableOpacity>
);
};

export default ServiceCard;

```

📁 ./src/hooks/useDebounce.ts

```

import { useState, useEffect } from 'react';

/**
 * @description
 * Um Custom Hook que implementa a lógica de "debouncing".
 * Ele recebe um valor (que muda rapidamente, como um texto de busca)
 * e um atraso em milissegundos. Ele retorna uma versão "atrasada" desse
 * valor,
 * que só é atualizada depois que o valor original para de mudar pelo tempo
 * do atraso.
 *

```

```
* @param {T} value 0 valor a ser "debounceado".
* @param {number} delay 0 atraso em milissegundos.
* @returns {T} 0 valor "debounceado".
*/
function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

export default useDebounce;
```